BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN MÔN HỌC

NHẬP MÔN KHOA HỌC DỮ LIỆU & HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

ĐỀ TÀI:

XÂY DỰNG BỘ DỮ LIỆU VỀ BẤT ĐỘNG SẢN, THỰC HIỆN CÁC PHƯƠNG PHÁP TỐI ƯU HIỆU NĂNG CHO HỆ THỐNG KHI DỮ LIỆU QUÁ LỚN

Sinh viên thực hiện: [Cần điền: Họ và tên sinh viên thực hiện]
Mã sinh viên: [Cần điền: Mã sinh viên / Lớp chuyên ngành]
Chuyên ngành: Khoa học Dữ liệu & Hệ thống Thông tin
Giảng viên hướng dẫn: [Cần điền: Học hàm, học vị và Họ tên Giảng viên hướng dẫn]

Hà Nội – 2026

TÓM TẮT TIỂU LUẬN (ABSTRACT)

Khi khối lượng dữ liệu giao dịch và thông tin bất động sản gia tăng nhanh chóng từ hàng chục nghìn đến hàng trăm nghìn và hàng triệu bản ghi, các hệ thống cơ sở dữ liệu quan hệ truyền thống và đường ống tiền xử lý dữ liệu phải đối mặt với thách thức nghiêm trọng về độ trễ truy vấn, nghẽn cổ chai I/O và áp lực bộ nhớ. Tiểu luận này thực hiện một nghiên cứu thực nghiệm có kiểm soát nhằm giải quyết đồng bộ ba bài toán: xây dựng bộ dữ liệu bất động sản thực tế chuẩn hóa, tối ưu hóa tầng nạp và xử lý dữ liệu tiền kỳ trong bộ nhớ, và đánh giá định lượng các kỹ thuật tối ưu hóa truy vấn trong hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL (phiên bản 18.3 x86_64).

Nội dung nghiên cứu được tổ chức chặt chẽ xoay quanh ba câu hỏi nghiên cứu cốt lõi (**RQ1 – RQ3**):

1. RQ1 (Xây dựng bộ dữ liệu và đánh giá chất lượng dữ liệu): Nghiên cứu thiết kế quy trình nạp luồng (streaming ingestion) kết hợp bộ đệm xáo trộn có kiểm soát (`seed = 42`, `buffer_size = 10,000`, `batch_size = 2,000`) từ kho dữ liệu Hugging Face (`Phuscca/vietnam-real-estates`) để trích xuất 500.000 bản ghi theo hợp đồng 9 thuộc tính chuẩn hóa. Kết quả kiểm toán và phân tích chất lượng dữ liệu (Data Profiling) ghi nhận: 6 trường thông tin bắt buộc đạt độ đầy đủ 100%; trường `district_name` thiếu đúng 14.235 bản ghi (2,85%); trường `ward_name` thiếu đúng 72.150 bản ghi (14,43%); trường giá chào bán (`price`) thiếu đúng 39.061 bản ghi (7,81%), phản ánh trung thực các tin đăng dạng giá thỏa thuận trên thị trường và được giữ nguyên giá trị SQL NULL. Phân bố địa phương tập trung chủ yếu tại TP. Hồ Chí Minh (186.698 tin, chiếm 37,34%), Hà Nội (170.167 tin, chiếm 34,03%) và Hải Phòng (10.422 tin, chiếm 2,08%). Do phân bố giá và diện tích lệch phải mạnh (skewed), nghiên cứu xác lập các đại lượng thống kê vững: trung vị diện tích là **80 m²** (khoảng tứ phân vị IQR = 64,1 m²) và trung vị giá chào bán là **7,4 tỷ VNĐ** (IQR = 10,2 tỷ VNĐ).

2. RQ2 (Đánh giá hiệu năng xử lý dữ liệu bằng Pandas, Polars, CSV, Parquet và Chunked Processing): Ở tầng tiền xử lý dữ liệu, thực nghiệm so sánh định dạng lưu trữ (CSV vs Parquet) và động cơ tính toán (Pandas vs Polars) kết hợp luồng giám sát tài nguyên RAM thực thời (lấy mẫu RSS 1 ms). Định dạng Parquet nhị phân nén hướng cột (dung lượng 41,42 MB) giúp tiết kiệm 55,9% không gian đĩa so với CSV (93,88 MB). Động cơ Polars đọc tệp CSV 500.000 dòng trong **75,27 ms**, nhanh gấp **40,3 lần** so với Pandas read_csv (**3.038,53 ms**). Trong tác vụ lọc và tính toán tổng hợp nội bộ (In-Memory Analytics), Polars hoàn thành trong **13,88 ms**, nhanh gấp 4,0 lần Pandas (55,17 ms). Đối với bài toán kiểm soát bộ nhớ, kỹ thuật nạp phân khối (Chunked Streaming với kích thước 10.000 dòng) giữ mức tăng bộ nhớ thực tế (Delta RSS) ở ngưỡng ổn định: thao tác nạp phân khối chỉ tăng **+19,68 MB** RAM (so với +241,37 MB khi nạp toàn bộ qua Pandas read_csv), và thao tác nạp kèm tổng hợp phân khối (Chunked Aggregate) có Delta RSS là **+21,09 MB**.

3. RQ3 (Đánh giá tối ưu PostgreSQL thông qua Q1–Q4): Trên hệ quản trị PostgreSQL 18.3, nghiên cứu thực hiện 600 lượt đo chính thức (3 quy mô 50K, 200K, 500K × 10 cấu hình × 20 lần chạy sau 3 lượt warm-up):

- *Với bài toán lọc đẳng thức đơn cột (Q1 - Hải Phòng, 10.422 bản ghi tại 500K):* Chỉ mục B-tree `idx_properties_province_name` chuyển đổi kế hoạch từ `Seq Scan` sang `Bitmap Index`

`Scan + Bitmap Heap Scan`, giảm số khối bộ đệm từ 12.808 xuống 7.236 Shared Hit Blocks, đạt hệ số tăng tốc **4,04 lần** tại 500K (và 8,21 lần tại 50K).

- *Với bài toán lọc khoảng, sắp xếp và cắt ngọn (Q2 - Cầu Giấy, Top 20 giá thấp nhất):* Chỉ mục tổ hợp Composite Index `(province_name, district_name, price, id)` kích hoạt cơ chế Early Termination, dừng duyệt ngay khi đủ 20 bản ghi, loại bỏ hoàn toàn nút `Sort` (Top-N Heapsort) trong kế hoạch thực thi, giảm số khối truy cập bộ đệm từ 12.898 xuống đúng **23 Shared Hit Blocks** (giảm 560 lần), đem lại mức tăng tốc **942,37 lần** tại 500K (từ 48,06 ms xuống 0,051 ms).
- *Với bài toán thời gian và tính khả chỉ mục (Q3 - ma trận trực giao 2 × 2):* Thực nghiệm chứng minh chỉ mục B-tree bị vô hiệu hóa hoàn toàn khi vị từ chứa hàm ép kiểu `published_at::date` (tỷ lệ sử dụng index là 0,00%, Speedup = 0,95x, lãng phí 10,73 MB index). Ngược lại, viết lại vị từ sang khoảng sargable kết hợp chỉ mục B-tree giúp chuyển sang Bitmap Index Scan, giảm 79,2% số khối đệm và tăng tốc **17,97 lần** tại 500K.
- *Với bài toán đối chứng âm và chi phí lưu trữ (Q4):* Truy vấn tổng hợp toàn bảng (`GROUP BY` không WHERE) duy trì kế hoạch `Seq Scan + HashAggregate` bất kể có chỉ mục hay không (Speedup = 0,95x), chứng minh giới hạn của B-tree khi phải đọc toàn bộ bảng. Tổng dung lượng của 4 chỉ mục đo riêng lẻ theo cấu hình tại quy mô 500K là **46,02 MB** (chiếm 45,6% dung lượng bảng Heap 100,8 MB).

Toàn bộ kế hoạch thực thi chi tiết được lưu trữ trong 600 tệp JSON kiểm toán bằng mã băm SHA-256, cùng bộ kiểm thử tự động xác nhận tính toàn vẹn và khả năng tái lập của nghiên cứu.

LỜI CẢM ƠN (ACKNOWLEDGEMENTS)

Nhóm nghiên cứu xin bày tỏ lòng biết ơn sâu sắc đến các Thầy/Cô bộ môn Khoa học Dữ liệu và Hệ thống Thông tin, Khoa Công nghệ Thông tin - Học viện Công nghệ Bưu chính Viễn thông đã tận tình truyền thụ nền tảng lý thuyết vững chắc về giải thuật, cấu trúc dữ liệu, nguyên lý hệ quản trị cơ sở dữ liệu và phương pháp luận nghiên cứu khoa học.

Nhóm nghiên cứu chân thành cảm ơn cộng đồng mã nguồn mở Hugging Face và tác giả bộ dữ liệu `Phuscca/vietnam-real-estates` đã cung cấp nguồn tài nguyên dữ liệu phong phú phục vụ quá trình học tập và nghiên cứu thực nghiệm.

Mặc dù nhóm nghiên cứu đã nỗ lực hết mình với tinh thần làm việc nghiêm túc, khoa học và cầu thị, tiểu luận khó tránh khỏi những giới hạn nhất định. Nhóm rất mong nhận được những nhận xét, góp ý và chỉ dẫn quý báu từ quý Thầy/Cô trong Hội đồng phản biện để tiếp tục hoàn thiện đề tài trong các giai đoạn phát triển tiếp theo.

MỤC LỤC (TABLE OF CONTENTS)

[Nhấn F9 hoặc chuột phải chọn 'Update Field' để cập nhật Mục lục tự động]

TÓM TẮT TIỂU LUẬN (ABSTRACT) 2
LỜI CẢM ƠN (ACKNOWLEDGEMENTS) 4
DANH MỤC THUẬT NGỮ VÀ VIẾT TẮT 8
DANH MỤC BẢNG BIỂU 10
DANH MỤC HÌNH VẼ 11
CHƯƠNG 1: MỞ ĐẦU 13
1.1. Bối cảnh và Đặt vấn đề 13
1.2. Mục tiêu nghiên cứu 14
1.3. Đối tượng và Phạm vi nghiên cứu 14
1.4. Các câu hỏi nghiên cứu chuẩn hóa (RQ1 – RQ3) 14
Câu hỏi nghiên cứu 1 (RQ1): Xây dựng bộ dữ liệu và đánh giá chất lượng dữ liệu 15
Câu hỏi nghiên cứu 2 (RQ2): Đánh giá hiệu năng xử lý dữ liệu bằng Pandas, Polars, CSV, Parquet và Chunked Processing 15
Câu hỏi nghiên cứu 3 (RQ3): Đánh giá tối ưu PostgreSQL thông qua Q1–Q4 15
1.5. Ý nghĩa khoa học và thực tiễn của đề tài 15
1.6. Bố cục của tiểu luận 15
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÀI LIỆU 17
2.1. Kiến trúc lưu trữ và Quản lý bộ đệm trong PostgreSQL 17
2.1.1. Cấu trúc bảng Heap và Định dạng Page 8 KB 17
2.1.2. Cơ chế Buffer Pool và Phân biệt Shared Hit Blocks với Shared Read Blocks 17
2.2. Cấu trúc dữ liệu và Cơ chế hoạt động của Chỉ mục B-tree 18
2.2.1. Cây B-tree đa nhánh và Phân tích chi phí tìm kiếm 18
2.2.2. Composite Index và Quy tắc tiền tố ngoài cùng bên trái (Leftmost Prefix Rule) 19
2.3. Bộ lập lịch truy vấn (Query Optimizer) và Mô hình chi phí (Cost Model) 20
2.3.1. Các phương thức quét bảng: Sequential Scan, Index Scan, Bitmap Heap Scan 20
2.3.2. Thuật toán sắp xếp trong bộ nhớ (Top-N Heapsort vs Loại bỏ nút Sort) 21
2.4. Tính khả chỉ mục của vị từ truy vấn (Sargability) 22
2.4.1. Khái niệm Sargable Predicate 22
2.4.2. Tác động của hàm ép kiểu và biểu thức trên cột chỉ mục 22
2.5. Lý thuyết xử lý dữ liệu lớn ở tầng tiền kỳ (In-memory Data Ingestion) 23

2.5.1. Định dạng hàng (CSV) đối chiếu định dạng cột nhị phân (Parquet) 23
2.5.2. Kỹ thuật nạp phân khối (Chunked Streaming) và Giới hạn đo lường RAM RSS . 23
CHƯƠNG 3: XÂY DỰNG BỘ DỮ LIỆU VÀ ĐÁNH GIÁ CHẤT LƯỢNG DỮ LIỆU 25
3.1. Quy trình thu thập dữ liệu dòng từ Hugging Face 25
3.1.1. Nguồn dữ liệu và Cơ chế Streaming Shuffle Buffer 25
3.1.2. Tính xác định và Giới hạn của Dataset Revision 25
3.2. Bộ quy tắc làm sạch dữ liệu và Hợp đồng 9 thuộc tính 26
3.2.1. Tiêu chí toàn vẹn và Xử lý giá trị khuyết thiếu nghiệp vụ 26
3.3. Lược đồ cơ sở dữ liệu quan hệ trên PostgreSQL 28
3.4. Kết quả Profiling và Phân tích khám phá dữ liệu (EDA) 28
3.4.1. Thống kê khuyết thiếu và Kiểm toán nạp dòng 28
3.4.2. Phân bố loại hình bất động sản và Địa bàn hành chính 30
3.4.3. Thống kê mô tả định lượng vững (Robust Statistics) 31
3.4.4. Xu hướng thời gian và Phân tích tương quan Spearman 34
3.4.5. Kiểm toán bản ghi trùng lặp 36
3.5. Kỹ thuật tạo đặc trưng và Dữ liệu làm giàu (Feature Engineering) 36
CHƯƠNG 4: PHƯƠNG PHÁP THỰC NGHIỆM 38
4.1. Môi trường thực nghiệm phần cứng và phần mềm 38
4.2. Thiết kế thực nghiệm RQ2: Đánh giá hiệu năng xử lý dữ liệu tiền kỳ 38
4.2.1. Kịch bản thực nghiệm Ingestion và Phân tích nội bộ 38
4.2.2. Phương pháp giám sát bộ nhớ thực thời và Giới hạn phép lấy mẫu RSS 1 ms 39
4.3. Thiết kế thực nghiệm RQ3: Tối ưu hóa truy vấn PostgreSQL 40
4.3.1. Ba quy mô dữ liệu thực nghiệm: 50.000, 200.000 và 500.000 bản ghi 40
4.3.2. Bốn bài toán truy vấn SQL thực nghiệm (Q1 – Q4) 40
4.3.3. Phân rã 10 cấu hình thực nghiệm Benchmark PostgreSQL 41
4.4. Giao thức đo lường kiểm soát nhiễu thực nghiệm 42
4.5. Thiết kế kiểm thử tính tương đương ngữ nghĩa truy vấn (Equivalence Verification) .. 43
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN KHOA HỌC 44
5.1. Đánh giá chất lượng và Tính đại diện của bộ dữ liệu (Trả lời RQ1) 44
5.2. Hiệu năng xử lý dữ liệu với Pandas, Polars, CSV, Parquet và Chunked Processing (Trả lời RQ2) 45
5.2.1. Tốc độ nạp tệp: Ưu thế vượt trội của Polars CSV và Định dạng Parquet 45
5.2.2. Phân tích kỹ thuật nạp phân khối Chunked Streaming và Mức tăng RAM 46
5.2.3. Hiệu năng lọc và Tổng hợp dữ liệu trong bộ nhớ 46

5.3. Đánh giá tối ưu hóa truy vấn PostgreSQL thông qua Q1 – Q4 (Trả lời RQ3) 46
5.3.1. Tổng quan bảng số liệu Benchmark qua 3 quy mô dữ liệu 46
5.3.2. Bài toán Q1: Hiệu quả của chỉ mục đơn cột và Chuyển đổi Bitmap Scan 50
5.3.3. Bài toán Q2: Đột phá hiệu năng Composite Index, Early Termination và Triệt tiêu nút Sort 51
5.3.4. Bài toán Q3: Phân tích trực giao 2x2, Vô hiệu hóa Index và Cộng hưởng Sargable 53
5.3.5. Bài toán Q4: Đối chứng âm Aggregation Control và Giới hạn của B-tree 54
5.3.6. Chi phí phụ trội lưu trữ vật lý của các chỉ mục B-tree (46,02 MB tại 500K) 54
5.4. Tổng hợp đối chiếu và Ma trận chứng tích thực nghiệm 56
CHƯƠNG 6: KẾT LUẬN VÀ KHUYẾN NGHỊ 57
6.1. Tổng kết các đóng góp chính của đề tài 57
6.2. Câu trả lời tường minh cho ba câu hỏi nghiên cứu (RQ1 – RQ3) 57
Trả lời Câu hỏi nghiên cứu 1 (RQ1) 58
Trả lời Câu hỏi nghiên cứu 2 (RQ2) 58
Trả lời Câu hỏi nghiên cứu 3 (RQ3) 58
6.3. Khuyến nghị kỹ thuật thực tiễn cho kỹ sư dữ liệu và DBA 59
6.4. Hạn chế của nghiên cứu 59
6.5. Định hướng phát triển tiếp theo 60
TÀI LIỆU THAM KHẢO 61
PHỤ LỤC 62
Phụ lục A: Lược đồ DDL và Ràng buộc toàn vẹn cơ sở dữ liệu 62
Phụ lục B: Tập mã nguồn các câu truy vấn Benchmark SQL chuẩn hóa 62
B.1. Bài toán Q1 (`sql/benchmark/queries/q1.sql`) 62
B.2. Bài toán Q2 (`sql/benchmark/queries/q2.sql`) 62
B.3. Bài toán Q3 Truy vấn gốc (`sql/benchmark/queries/q3_original.sql`) 62
B.4. Bài toán Q3 Truy vấn viết lại (`sql/benchmark/queries/q3_rewritten.sql`) 63
B.5. Bài toán Q4 (`sql/benchmark/queries/q4.sql`) 63
Phụ lục C: Bảng kê mã băm SHA-256 các tệp kế hoạch thực thi thô 63

DANH MỤC THUẬT NGỮ VÀ VIẾT TẮT

Viết tắt / Thuật ngữ	Tiếng Anh đầy đủ	Giải thích ngữ nghĩa
CBO	Cost-Based Optimizer	Bộ tối ưu hóa truy vấn dựa trên mô hình chi phí ước tính
CSV	Comma-Separated Values	Định dạng tệp văn bản thuần lưu trữ bản ghi theo từng dòng
CTE	Common Table Expression	Mệnh đề WITH định nghĩa bảng tạm thời trong câu truy vấn SQL
DDL	Data Definition Language	Tập lệnh định nghĩa cấu trúc cơ sở dữ liệu (CREATE, ALTER, DROP)
DML	Data Manipulation Language	Tập lệnh thao tác dữ liệu (INSERT, UPDATE, DELETE)
EDA	Exploratory Data Analysis	Quá trình phân tích khám phá cấu trúc và đặc trưng của dữ liệu
IQR	Interquartile Range	Khoảng tứ phân vị (IQR = Q_3 - Q_1), thước đo độ phân tán vững
OOM	Out Of Memory	Lỗi tiến trình bị hệ điều hành cưỡng chế dừng do cạn kiệt RAM
Parquet	Apache Parquet	Định dạng tệp lưu trữ nhị phân hướng cột có nén và siêu dữ liệu
PK	Primary Key	Khóa chính định danh duy nhất cho từng bản ghi trong bảng
Q1 – Q4	Query 1 to Query 4	Bốn bài toán truy vấn SQL điển hình được thiết kế cho thực nghiệm
RQ1 – RQ3	Research Questions 1 to 3	Ba câu hỏi nghiên cứu chuẩn hóa định hướng đề tài
RSS	Resident Set Size	Dung lượng bộ nhớ RAM vật lý thực tế của một tiến trình hệ thống
Sargable	Search Argument Able	Tính chất của vị từ cho phép tận dụng cây chỉ mục B-tree
Seq Scan	Sequential Scan	Phương thức quét tuần tự toàn bộ các trang dữ liệu trong bảng Heap
Shared Buffers	PostgreSQL Shared Buffers	Vùng bộ đệm RAM chia sẻ của PostgreSQL lưu trữ các trang 8 KB
TID	Tuple Identifier	Con trỏ vật lý trỏ trực tiếp đến vị trí hàng trong bảng (Block, Offset)

DANH MỤC BẢNG BIỂU

Số hiệu	Tên bảng biểu	Vị trí
Bảng 2.1	Ma trận tiêu chí lựa chọn phương thức quét bảng của Bộ tối ưu hóa chi phí (CBO)	Chương 2
Bảng 3.1	Đặc tả cấu trúc lược đồ dữ liệu chuẩn hóa (9 thuộc tính)	Chương 3
Bảng 3.2	Kết quả kiểm toán quá trình thu thập và lọc dòng dữ liệu (500K)	Chương 3
Bảng 3.3	Thống kê mô tả định lượng vững trên tập dữ liệu 500.000 bản ghi	Chương 3
Bảng 4.1	Đặc tả 10 cấu hình thực nghiệm Benchmark PostgreSQL	Chương 4
Bảng 5.1	Kết quả đo lường hiệu năng nạp và xử lý dữ liệu tiền kỳ (500.000 bản ghi)	Chương 5
Bảng 5.2	Tổng hợp kết quả đo đạc Benchmark PostgreSQL trên 3 quy mô dữ liệu	Chương 5
Bảng 5.3	Diễn biến hiệu năng của bài toán Q1 qua 3 thang đo dữ liệu	Chương 5
Bảng 5.4	So sánh chi tiết hiệu năng bài toán Q2 và Cơ chế Early Termination	Chương 5
Bảng 5.5	Bảng phân rã trực giao 2x2 hiệu năng bài toán Q3 qua 3 thang đo	Chương 5
Bảng 5.6	Kết quả thực nghiệm bài toán đối chứng âm Q4 qua 3 thang đo	Chương 5
Bảng 5.7	Chi phí phụ trội lưu trữ vật lý của các chỉ mục B-tree qua các thang đo	Chương 5

DANH MỤC HÌNH VẼ

Số hiệu	Tên hình vẽ	Vị trí	Tệp ảnh nguồn
Hình 3.1	Tỷ lệ dữ liệu khuyết thiếu trên các thuộc tính bất động sản	Chương 3	`results/eda/missing_values_bar.png`
Hình 3.2	Phân bố số lượng tin đăng theo loại hình bất động sản	Chương 3	`results/eda/property_type_count.png`
Hình 3.3	Top các tỉnh/thành phố có mật độ tin đăng lớn nhất Việt Nam	Chương 3	`results/eda/province_distribution.png`
Hình 3.4	Phân bố giá chào bán bất động sản (Thang đo Logarithmic)	Chương 3	`results/eda/price_distribution.png`
Hình 3.5	Phân bố diện tích bất động sản (Thang đo Logarithmic)	Chương 3	`results/eda/area_distribution.png`
Hình 3.6	Phân bố giá chào bán theo từng loại hình bất động sản (Boxplot)	Chương 3	`results/eda/price_by_property_type_boxplot.png`
Hình 3.7	Đơn giá chào bán trung vị (Triệu/m²) tại các địa phương trọng điểm	Chương 3	`results/eda/price_per_m2_by_province.png`
Hình 3.8	Mật độ tương quan giữa giá và diện tích bất động sản (Hexbin Plot)	Chương 3	`results/eda/price_vs_area_scatter.png`
Hình 3.9	Xu hướng số lượng tin đăng bất động sản theo thời gian đăng tin	Chương 3	`results/eda/monthly_listings_trend.png`
Hình 3.10	Ma trận hệ số tương quan phi tuyến Spearman giữa các biến định lượng	Chương 3	`results/eda/numeric_correlation_heatmap.png`

Số hiệu	Tên hình vẽ	Vị trí	Tệp ảnh nguồn
Hình 5.1	Thời gian thực thi trung vị theo quy mô dữ liệu của 4 bài toán truy vấn	Chương 5	`results/benchmark/analysis/execution_time_by_scale.png`
Hình 5.2	So sánh hệ số tăng tốc (Speedup Factor) của các phương pháp tối ưu	Chương 5	`results/benchmark/analysis/speedup_comparison.png`
Hình 5.3	Chi phí phụ trội lưu trữ (Storage Overhead) của các chỉ mục B-tree	Chương 5	`results/benchmark/analysis/index_storage_overhead.png`

CHƯƠNG 1: MỞ ĐẦU

1.1. Bối cảnh và Đặt vấn đề

Trong kỷ nguyên kinh tế số, thị trường bất động sản là một trong những lĩnh vực có tốc độ tích lũy và luân chuyển dữ liệu nhanh nhất. Các cổng thông tin niêm yết thương mại điện tử, hệ thống thẩm định giá tự động và ứng dụng quản lý danh mục tài sản phải tiếp nhận hàng triệu tin đăng bất động sản với đặc thù đa chiều phức tạp: vừa chứa dữ liệu văn bản phi cấu trúc (tiêu đề, mô tả), dữ liệu phân cấp hành chính (tỉnh/thành phố, quận/huyện, phường/xã), vừa chứa dữ liệu định lượng có biên độ biến động rất rộng (diện tích, giá bán từ vài triệu đến hàng trăm tỷ đồng) và chuỗi dấu thời gian liên tục (`published_at`).

Khi quy mô bảng dữ liệu gia tăng từ hàng chục nghìn lên hàng trăm nghìn và hàng triệu dòng, các truy vấn phục vụ người dùng cuối (như lọc bất động sản theo địa phương, tìm kiếm trong khoảng giá, sắp xếp theo giá tăng dần hoặc tổng hợp báo cáo thị trường theo phân khúc) bộc lộ các vấn đề nghẽn cổ chai hiệu năng nghiêm trọng:

1. Chi phí I/O quét toàn bộ bảng (Sequential Scan Overhead): Khi không có cơ chế chỉ mục phù hợp, hệ quản trị cơ sở dữ liệu buộc phải quét tuần tự toàn bộ các trang dữ liệu trong bảng Heap. Thời gian phản hồi của hệ thống tăng theo quy mô kích thước bảng, gây suy giảm năng lực phục vụ của máy chủ trong các khung giờ cao điểm.

2. Chi phí sắp xếp trong bộ nhớ (In-memory Sorting Overhead): Với các câu truy vấn có mệnh đề sắp xếp và cắt ngọn (`ORDER BY ... LIMIT K`), nếu hệ thống không thể tận dụng cấu trúc chỉ mục để đọc dữ liệu theo thứ tự đã được sắp sẵn, nó phải nạp toàn bộ tập kết quả thỏa mãn điều kiện lọc vào RAM và thực thi các giải thuật sắp xếp (như Top-N Heapsort hoặc External Merge Sort), tiêu tốn chu kỳ CPU và dung lượng vùng nhớ tạm thời (`work_mem`).

3. Hiểu nhầm phổ biến về Đánh chỉ mục (Indexing Misconceptions): Trong thực tế phát triển phần mềm, tồn tại xu hướng tiếp cận cảm tính: *"Cứ thấy truy vấn chậm là tạo thêm chỉ mục B-tree trên các cột trong mệnh đề WHERE"*. Cách tiếp cận này bỏ qua nguyên lý hoạt động của bộ tối ưu hóa chi phí (Cost-Based Optimizer), không phân biệt được vị từ khả chỉ mục (sargable) với vị từ không khả chỉ mục (non-sargable), và không lường trước được chi phí phụ trội lưu trữ vật lý (Storage Overhead) cũng như độ trễ bảo trì chỉ mục khi ghi dữ liệu (Write Penalty).

4. Nghẽn cổ chai ở tầng tiền xử lý dữ liệu (Data Preprocessing Pipeline): Trước khi dữ liệu được nạp vào cơ sở dữ liệu quan hệ, các tác vụ trích xuất, làm sạch và tổng hợp dữ liệu thường gặp hiện tượng tràn bộ nhớ (Out-Of-Memory - OOM) khi nạp các tệp dữ liệu văn bản lớn bằng các thư viện phân tích truyền thống như Pandas trên các môi trường máy trạm có tài nguyên bộ nhớ hữu hạn.

Xuất phát từ những thách thức lý thuyết và thực tiễn nêu trên, đề tài **"Xây dựng bộ dữ liệu về bất động sản, thực hiện các phương pháp tối ưu hiệu năng cho hệ thống khi dữ liệu quá lớn"** được triển khai nhằm xây dựng một mô hình thực nghiệm chặt chẽ, định lượng hóa một cách khách quan ảnh hưởng của các kỹ thuật tối ưu hóa trong hệ quản trị cơ sở dữ liệu PostgreSQL (phiên bản 18.3 x86_64) và các công cụ xử lý dữ liệu hiện đại.

1.2. Mục tiêu nghiên cứu

Đề tài hướng tới ba mục tiêu cụ thể:

1. Xây dựng quy trình xử lý dữ liệu chuẩn tắc (Data Engineering Pipeline): Thu thập luồng dữ liệu từ nguồn thực tế Hugging Face, áp dụng các quy tắc làm sạch chuẩn hóa, trích xuất hợp đồng 9 thuộc tính cốt lõi và xây dựng ba tập dữ liệu ở các quy mô 50.000, 200.000 và 500.000 bản ghi với các tham số kiểm soát xác định.

2. Đánh giá hiệu năng xử lý dữ liệu tiền kỳ (In-memory & File Processing): Khảo sát định lượng ảnh hưởng của định dạng lưu trữ (CSV vs Parquet), động cơ tính toán (Pandas vs Polars) và kỹ thuật nạp phân khối (Chunked Streaming) đến thời gian xử lý và mức chiếm dụng tài nguyên bộ nhớ thực tế (Resident Set Size - RSS).

3. Đánh giá thực nghiệm các kỹ thuật tối ưu hóa trong PostgreSQL (Relational Optimization): Thiết kế khung thực nghiệm có kiểm soát (cô lập trạng thái bộ đệm với 3 lượt warm-up, 20 lượt đo chính thức lấy trung vị và khoảng tứ phân vị IQR, trích xuất toàn bộ kế hoạch thực thi chi tiết `EXPLAIN (ANALYZE, BUFFERS)`) để làm rõ hiệu quả và giới hạn của chỉ mục B-tree đơn cột, Composite Index bao phủ sắp xếp/cắt ngọn, kỹ thuật viết lại truy vấn (Query Rewriting / Sargability), bài toán đối chứng âm (Negative Control) và chi phí lưu trữ phụ trội.

1.3. Đối tượng và Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:**
- Bộ dữ liệu bất động sản Việt Nam (`Phuscca/vietnam-real-estates`) chứa thông tin địa lý, phân loại tài sản, giá cả và thời gian đăng tin.
- Các kỹ thuật tối ưu hóa cơ sở dữ liệu: Single-column Index, Composite Index, Query Rewriting (Sargability), Aggregation Control.
- Các cơ chế nội tại của PostgreSQL: Quản lý bộ đệm (Shared Buffers), các kiểu quét dữ liệu (Seq Scan, Index Scan, Bitmap Index Scan, Bitmap Heap Scan), giải thuật sắp xếp (Top-N Heapsort).
- Kiến trúc xử lý dữ liệu tiền kỳ: Định dạng tệp CSV và Parquet; Thư viện phân tích Pandas và Polars; Kỹ thuật nạp phân khối (Chunked Streaming).
- **Phạm vi nghiên cứu:**
- Quy mô dữ liệu: Khảo sát tại 3 ngưỡng thực nghiệm: 50.000, 200.000 và 500.000 bản ghi.
- Hệ quản trị cơ sở dữ liệu: PostgreSQL 18.3 chạy trên hệ điều hành Windows x86_64, cấu hình máy trạm 16 GB RAM, ổ cứng SSD NVMe.
- Điều kiện đo đạc: Đo lường hiệu năng truy vấn đơn tiến trình (Single-client benchmarking) trong trạng thái hệ thống ổn định, tập trung vào thời gian lập kế hoạch (`Planning Time`), thời gian thực thi (`Execution Time`), số khối bộ đệm truy xuất (`Shared Hit Blocks`) và mức chiếm dụng bộ nhớ RAM tiến trình (`Delta RSS`).

1.4. Các câu hỏi nghiên cứu chuẩn hóa (RQ1 – RQ3)

Để định hướng toàn bộ quá trình thiết kế thực nghiệm và phân tích dữ liệu, đề tài xây dựng ba câu hỏi nghiên cứu chuẩn hóa:

Câu hỏi nghiên cứu 1 (RQ1): Xây dựng bộ dữ liệu và đánh giá chất lượng dữ liệu

> ***RQ1:*** *Làm thế nào để xây dựng một quy trình nạp luồng và làm sạch dữ liệu lớn có thể kiểm soát được từ nguồn bất động sản thực tế, và đặc trưng chất lượng dữ liệu (tỷ lệ khuyết thiếu, phân bố không gian, ngoại lệ định lượng và độ tin cậy) của tập dữ liệu này thể hiện như thế nào ở các quy mô từ 50.000 đến 500.000 bản ghi?*

Câu hỏi nghiên cứu 2 (RQ2): Đánh giá hiệu năng xử lý dữ liệu bằng Pandas, Polars, CSV, Parquet và Chunked Processing

> ***RQ2:*** *Ở tầng tiền xử lý dữ liệu, sự kết hợp giữa định dạng lưu trữ (CSV vs Parquet) và động cơ xử lý (Pandas vs Polars) tác động như thế nào đến tốc độ nạp tệp, hiệu năng tính toán phân tích và mức chiếm dụng tài nguyên bộ nhớ thực tế (Delta RSS); đồng thời kỹ thuật nạp phân khối (Chunked Streaming) đóng vai trò gì trong việc kiểm soát đỉnh bộ nhớ?*

Câu hỏi nghiên cứu 3 (RQ3): Đánh giá tối ưu PostgreSQL thông qua Q1–Q4

> ***RQ3:*** *Khi quy mô dữ liệu mở rộng đến 500.000 bản ghi trong PostgreSQL, các kỹ thuật tối ưu hóa cơ sở dữ liệu quan hệ (Chỉ mục B-tree đơn cột trong Q1, Composite Index bao phủ lọc/sắp xếp/cắt ngọn trong Q2, Viết lại truy vấn sargable trong Q3) mang lại hiệu quả tăng tốc và thay đổi cơ chế truy cập bộ đệm (Shared Hit Blocks) như thế nào, đâu là giới hạn của B-tree trong bài toán đối chứng âm Q4, và chi phí phụ trội lưu trữ vật lý của các chỉ mục là bao nhiêu?*

1.5. Ý nghĩa khoa học và thực tiễn của đề tài

- **Ý nghĩa khoa học:**
- Cung cấp một bộ dữ liệu thực nghiệm có kiểm chứng (600 tệp JSON kế hoạch thực thi chi tiết) minh chứng cho quy luật hoạt động của bộ tối ưu hóa chi phí trong hệ quản trị cơ sở dữ liệu hiện đại (PostgreSQL 18.3).
- Phân tách độc lập các biến số ảnh hưởng (Sargability đối chiếu Indexing; Lọc đẳng thức đối chiếu Lọc khoảng và Thứ tự sắp xếp; Lưu trữ hàng đối chiếu Lưu trữ cột) thông qua thiết kế thực nghiệm có kiểm soát.
- Thiết lập ma trận truy vết chứng tích khoa học (Evidence Matrix) kết nối chặt chẽ từ câu hỏi nghiên cứu, giả thuyết đến kết quả đo đạc định lượng cụ thể.
- **Ý nghĩa thực tiễn:**
- Đưa ra khuyến nghị thiết kế chỉ mục thực tế cho kỹ sư dữ liệu: thứ tự đặt cột trong Composite Index để triệt tiêu nút Sort, cách chuẩn hóa vị từ SQL tránh vô hiệu hóa index do ép kiểu, và giải pháp thay thế cho việc đánh chỉ mục trên truy vấn tổng hợp toàn bảng.
- Cung cấp quy trình xử lý dữ liệu theo lô (Chunking Pipeline) và khuyến nghị ứng dụng định dạng Parquet kết hợp Polars để tối ưu hóa thời gian và tài nguyên bộ nhớ trong các đường ống kỹ nghệ dữ liệu (Data Pipelines).
- Toàn bộ mã nguồn, dữ liệu thực nghiệm và công cụ trực quan hóa được lưu trữ minh bạch, phục vụ học tập và nghiên cứu thực nghiệm trong các môn học Khoa học Dữ liệu và Cơ sở dữ liệu.

1.6. Bố cục của tiểu luận

Tiểu luận được cấu trúc thành 6 chương chính theo mạch nghiên cứu khoa học:

- **Chương 1: Mở đầu** — Giới thiệu bối cảnh, mục tiêu, đối tượng, phạm vi, ba câu hỏi nghiên cứu chuẩn hóa (RQ1 – RQ3), ý nghĩa khoa học và thực tiễn của đề tài.
- **Chương 2: Cơ sở lý thuyết và Tổng quan tài liệu** — Trình bày nền tảng lý thuyết về kiến trúc bảng Heap, cơ chế Shared Buffers, cấu trúc chỉ mục B-tree, mô hình chi phí của bộ lập lịch truy vấn, nguyên lý Sargability và kiến trúc lưu trữ hướng cột.
- **Chương 3: Xây dựng bộ dữ liệu và Đánh giá chất lượng dữ liệu** — Trình bày quy trình thu thập dữ liệu dòng từ Hugging Face, bộ quy tắc làm sạch, lược đồ quan hệ PostgreSQL và kết quả hồ sơ chất lượng dữ liệu (Data Profiling & EDA), trực tiếp trả lời RQ1.
- **Chương 4: Phương pháp thực nghiệm** — Mô tả môi trường thực nghiệm, thiết kế kịch bản benchmark tiền xử lý cho RQ2 và thiết kế 10 cấu hình benchmark PostgreSQL (Q1 – Q4) cho RQ3, giao thức đo lường kiểm soát nhiễu và kiểm chứng tính tương đương ngữ nghĩa.
- **Chương 5: Kết quả thực nghiệm và Thảo luận khoa học** — Trình bày toàn bộ số liệu đo đạc thực tế, phân tích kế hoạch thực thi (EXPLAIN plan), phân tích buffer I/O, thời gian và tài nguyên RAM, trả lời dứt khoát RQ2 và RQ3.
- **Chương 6: Kết luận và Khuyến nghị** — Tổng kết các phát hiện khoa học, đưa ra khuyến nghị thực tiễn cho kỹ sư hệ thống, nêu rõ các hạn chế và định hướng phát triển tiếp theo của đề tài.
- **Tài liệu tham khảo & Phụ lục** — Liệt kê các tài liệu khoa học trích dẫn, lược đồ DDL, mã truy vấn benchmark và bảng kê chứng tích thực nghiệm SHA-256.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÀI LIỆU

2.1. Kiến trúc lưu trữ và Quản lý bộ đệm trong PostgreSQL

2.1.1. Cấu trúc bảng Heap và Định dạng Page 8 KB

Hệ quản trị cơ sở dữ liệu PostgreSQL tổ chức lưu trữ các bảng quan hệ dưới dạng các tệp bảng đống (**Heap Files**) không có thứ tự vật lý cố định. Mỗi bảng trên đĩa được chia thành các khối dữ liệu cố định gọi là các trang đệm (**Pages** hoặc **Blocks**) với kích thước mặc định là **8.192 bytes (8 KB)**.

Cấu trúc bên trong một trang 8 KB của PostgreSQL bao gồm các thành phần:

1. PageHeaderData (24 bytes): Chứa siêu dữ liệu của trang như con trỏ tới vùng dữ liệu trống tiếp theo (`pd_lower`), con trỏ tới đuôi của vùng dữ liệu (`pd_upper`), và thông tin kiểm tra tính toàn vẹn (checksum).

2. Mảng ItemIdData (Line Pointers): Là một mảng các con trỏ dòng (mỗi con trỏ chiếm 4 bytes) tăng dần từ đầu trang xuống. Mỗi con trỏ chứa thông tin độ dịch chuyển (offset) và độ dài trỏ tới vị trí bắt đầu của bản ghi dữ liệu thực tế bên dưới trang.

3. Vùng không gian trống (Free Space): Vùng đệm nằm giữa mảng con trỏ dòng và các bản ghi dữ liệu, sẽ thu hẹp dần khi có thêm các dòng mới được chèn vào.

4. Bản ghi dữ liệu (Items / Tuples): Chứa dữ liệu của các hàng, được lấp đầy từ đáy trang (offset 8191) ngược dần lên trên. Mỗi hàng có phần đầu bản ghi (**HeapTupleHeaderData**) chiếm tối thiểu 23 bytes chứa các thông tin kiểm soát phiên bản đa chiều (MVCC - Multiversion Concurrency Control) như `xmin`, `xmax`, `t_ctid`, và bitmap đánh dấu các giá trị `NULL`.

Mỗi hàng trong bảng Heap được định danh duy nhất bởi một con trỏ vật lý gọi là **TID (Tuple Identifier)** hoặc **ItemPointer**, có cấu trúc `(BlockNumber, OffsetNumber)`. Trong đó, `BlockNumber` là số thứ tự trang trong tệp bảng và `OffsetNumber` là chỉ số của con trỏ dòng trong trang đó.

> *QUAN TRỌNG:*
> *Do các hàng trong bảng Heap được chèn vào bất kỳ trang nào còn chỗ trống, việc truy xuất dữ liệu mà không có chỉ mục buộc PostgreSQL phải đọc tuần tự từ Block số 0 đến Block cuối cùng. Khóa chính (Primary Key) trong PostgreSQL mặc định tạo ra một cây chỉ mục B-tree duy nhất trên cột khóa, nhưng **hoàn toàn không tự động sắp xếp vật lý các hàng trong bảng Heap** trên đĩa theo thứ tự khóa chính (khác biệt căn bản với Clustered Index trong Microsoft SQL Server hoặc MySQL InnoDB).*

2.1.2. Cơ chế Buffer Pool và Phân biệt Shared Hit Blocks với Shared Read Blocks

Để giảm thiểu độ trễ cơ học và điện tử khi truy xuất đĩa cứng (Disk I/O), PostgreSQL duy trì một vùng nhớ đệm chia sẻ gọi là **Shared Buffers** trong RAM. Khi một câu lệnh truy vấn cần dữ liệu từ một trang, luồng xử lý (backend process) sẽ thực hiện các bước:

1. Tra cứu bảng băm bộ đệm (**Buffer Mapping Table**) xem trang `(relfilenode, forknum, blocknum)` đã có sẵn trong Shared Buffers hay chưa.

2. Nếu trang đã có sẵn trong RAM: ghi nhận một sự kiện **Shared Hit Block** (truy cập bộ đệm trúng đích). Đây là các lần truy xuất trang dữ liệu (8 KB) được phục vụ trực tiếp từ vùng nhớ đệm tuần hoàn Shared Buffers trong RAM, **hoàn toàn không phải là số block đọc vật lý từ ổ đĩa**. Tiến trình tăng số đếm sử dụng (`usage_count`) của trang đó theo thuật toán quét đồng hồ (**Clock Sweep** - biến thể hiệu năng cao của LRU) và đọc trực tiếp từ bộ nhớ với độ trễ cỡ nano-giây.

3. Nếu trang chưa có trong RAM: chọn một khung đệm (buffer frame) nạn nhân theo giải thuật Clock Sweep, đẩy trang cũ xuống đĩa nếu trang đó bị bẩn (`dirty page`), sau đó đọc trang 8 KB từ tệp trên đĩa cứng vào khung đệm, ghi nhận một sự kiện **Shared Read Block** (đọc khối đệm vật lý từ đĩa) với độ trễ cỡ mili-giây.

Trong các thực nghiệm đo lường hiệu năng chuyên sâu, chỉ số `Shared Hit Blocks` và `Shared Read Blocks` được trích xuất từ câu lệnh `EXPLAIN (ANALYZE, BUFFERS)` là thước đo định lượng khách quan nhất phản ánh chính xác khối lượng công việc I/O mà hệ thống phải gánh chịu. Khi hệ thống đã qua các lượt warm-up và dữ liệu nằm trọn trong RAM, `Shared Read Blocks` bằng 0 và toàn bộ công việc truy xuất được phản ánh qua `Shared Hit Blocks`.

2.2. Cấu trúc dữ liệu và Cơ chế hoạt động của Chỉ mục B-tree

2.2.1. Cây B-tree đa nhánh và Phân tích chi phí tìm kiếm

Chỉ mục mặc định và phổ biến nhất trong PostgreSQL là **B-tree (Balanced Tree)**, cụ thể là biến thể **B+ tree** được thiết kế tối ưu cho lưu trữ khối. Cây B-tree trong PostgreSQL là một cây tìm kiếm đa nhánh tự cân bằng, trong đó tất cả các nút lá (Leaf Nodes) đều nằm ở cùng một độ sâu.

Cấu trúc hình học của cây B-tree bao gồm:

- **Nút gốc (Root Node):** Nằm ở đỉnh cây, chứa các khóa phân cách hướng dẫn đường đi xuống các nút con.
- **Các nút trung gian (Internal Nodes):** Chứa các cặp `(Key, Child_BlockNumber)`, chia không gian khóa thành các khoảng rời rạc. Mỗi nút có hệ số phân nhánh (**Fan-out**) rất lớn, thường chứa từ hàng trăm đến hàng nghìn phần tử trên một trang 8 KB.
- **Các nút lá (Leaf Nodes):** Chứa các cặp `(IndexKey, TID)`. Trong đó `IndexKey` là giá trị thuộc tính được đánh chỉ mục và `TID` là con trỏ vật lý trỏ trực tiếp về trang Heap chứa hàng dữ liệu thực tế.
- **Liên kết hai chiều (Bidirectional Sibling Pointers):** Các nút lá ở cùng tầng đáy được liên kết chặt chẽ với nhau bằng con trỏ trái và phải (`bpt_left`, `bpt_right`). Đặc tính này cho phép hệ thống thực hiện các phép quét khoảng (**Range Scan**) hoặc duyệt theo thứ tự sắp xếp (**Ordered Scan**) cực kỳ hiệu quả mà không cần quay lui lên các nút cha.

Phân tích chi phí tìm kiếm (Search Cost Analysis):

Không thể khẳng định Index Scan có độ phức tạp thời gian cố định O(1) trong mọi trường hợp. Về mặt lý thuyết cấu trúc dữ liệu và giải thuật, chi phí tìm kiếm trong cây B-tree phụ thuộc vào:

1. Chiều cao của cây (h): Để định vị khóa lá đầu tiên thỏa mãn vị từ tìm kiếm từ nút gốc, hệ thống cần duyệt qua h tầng của cây:

$$h \approx \lceil \log_B (N / C_leaf) \rceil$$

Trong đó B là hệ số phân nhánh trung bình của nút trung gian, N là tổng số bản ghi và C_leaf là dung lượng chứa khóa của nút lá. Tại quy mô 500.000 bản ghi trong bài toán thực nghiệm, chiều cao cây B-tree h ≈ 3. Mỗi tầng tương ứng với một lần truy xuất trang chỉ mục (thường là 3 block truy cập).

2. Số lượng bản ghi cần truy xuất (k): Sau khi chạm đến nút lá đầu tiên, nếu câu truy vấn cần lấy k bản ghi, hệ thống phải duyệt tuần tự qua các trang lá lân cận và thực hiện các phép truy xuất trang Heap tương ứng để lấy dữ liệu chi tiết.

Do đó, tổng chi phí I/O và thời gian của Index Scan tổng quát có độ phức tạp là O(h + k), không phải O(1). Chỉ trong trường hợp bài toán có giới hạn cắt ngọn cố định rất nhỏ (k = const, ví dụ k = 20 trong truy vấn phân trang `LIMIT 20`) và các khóa đã được sắp xếp sẵn trên cây, chi phí duyệt lá mới bị chặn trên bởi một hằng số nhỏ độc lập với kích thước tổng thể N của bảng dữ liệu.

2.2.2. Composite Index và Quy tắc tiền tố ngoài cùng bên trái (Leftmost Prefix Rule)

Chỉ mục tổ hợp đa cột (**Composite Index**) là chỉ mục được xây dựng trên hai hoặc nhiều thuộc tính: `CREATE INDEX idx ON table (col1, col2, col3, ...)`.

Trong cấu trúc nội tại của cây B-tree đa cột, các khóa được sắp xếp theo thứ tự từ điển (lexicographical ordering):

1. Trước hết sắp xếp tuyệt đối theo giá trị của `col1`.
2. Nếu các giá trị `col1` trùng nhau, tiến hành sắp xếp tiếp theo giá trị của `col2`.
3. Nếu cả `col1` và `col2` đều trùng nhau, sắp xếp tiếp theo `col3`.

Từ nguyên lý sắp xếp từ điển này, quy tắc **Tiền tố ngoài cùng bên trái (Leftmost Prefix Rule)** được xác lập:

- Chỉ mục có thể phục vụ tối ưu cho các truy vấn lọc trên: `(col1)`, `(col1, col2)` hoặc `(col1, col2, col3)`.
- Chỉ mục **không thể sử dụng** để tìm kiếm trực tiếp trên `col2` hoặc `col3` nếu thiếu vắng điều kiện đẳng thức trên `col1`, bởi vì tại tầng lá, các giá trị của `col2` bị phân tán ngẫu nhiên qua các nhánh khác nhau của `col1`.

Đặc biệt, khi truy vấn kết hợp giữa phép lọc đẳng thức (`col1 = 'A'`), phép lọc khoảng (`col2 BETWEEN X AND Y`), phép sắp xếp (`ORDER BY col2, col3`) và cắt ngọn (`LIMIT K`), nếu chỉ mục được sắp xếp theo đúng thứ tự:

$$Index\ Columns = (Equality_Cols, Range_or_Sort_Cols, Tie_breaker_Cols)$$

thì toàn bộ các bản ghi thỏa mãn điều kiện lọc và thứ tự sắp xếp sẽ nằm liên tục cạnh nhau trên các trang lá của cây B-tree. Hệ thống có thể duyệt trực tiếp và dừng quét ngay khi gom đủ K bản ghi thỏa mãn (**Early Termination**), triệt tiêu hoàn toàn chi phí quét các bản ghi còn lại và chi phí sắp xếp trong bộ nhớ.

2.3. Bộ lập lịch truy vấn (Query Optimizer) và Mô hình chi phí (Cost Model)

2.3.1. Các phương thức quét bảng: Sequential Scan, Index Scan, Bitmap Heap Scan

Trước khi thực thi một câu lệnh SQL, bộ tối ưu hóa chi phí (Cost-Based Optimizer - CBO) của PostgreSQL tạo ra nhiều cây kế hoạch thực thi ứng viên (Execution Plans) và ước tính tổng chi phí theo mô hình hàm:

$$Cost = N_page_seq \cdot seq_page_cost + N_page_rand \cdot random_page_cost + N_tuples \cdot cpu_tuple_cost + N_ops \cdot cpu_operator_cost$$

Các tham số mặc định trong PostgreSQL:

- `seq_page_cost = 1.0`: Chi phí ước tính cho việc đọc tuần tự 1 trang 8 KB từ đĩa.
- `random_page_cost = 4.0`: Chi phí ước tính cho việc đọc ngẫu nhiên 1 trang 8 KB từ đĩa (phản ánh độ trễ tìm kiếm cơ học của ổ HDD truyền thống; trên SSD tham số này thường được cấu hình từ 1.1 đến 1.5).
- `cpu_tuple_cost = 0.01`: Chi phí CPU để xử lý một hàng dữ liệu.
- `cpu_operator_cost = 0.0025`: Chi phí CPU để kiểm tra một toán tử điều kiện trong mệnh đề WHERE.

Bộ lập lịch sẽ so sánh chi phí giữa ba phương thức quét bảng cơ bản (được tổng hợp trong Bảng 2.1):

Bảng 2.1: Ma trận tiêu chí lựa chọn phương thức quét bảng của Bộ tối ưu hóa chi phí (PostgreSQL CBO)

Phương thức quét	Điều kiện Index & Độ chọn lọc (Selectivity)	Cơ chế truy cập I/O	Kịch bản áp dụng tối ưu
Sequential Scan (Seq Scan)	Không có Index phù hợp HOẶC độ chọn lọc thấp (> 15–20% tổng số dòng)	Đọc tuần tự toàn bộ các trang Heap 8 KB từ đầu đến cuối bảng	Bảng có kích thước nhỏ, hoặc câu truy vấn cần nạp phần lớn dữ liệu bảng
Index Scan	Có Index khả dụng, độ chọn lọc cực cao (< 1–2%), cần lấy ít dòng	Duyệt B-tree lấy TID, sau đó đọc ngẫu nhiên (Random I/O) trực tiếp vào trang Heap	Truy vấn tìm kiếm theo khóa chính/duy nhất, hoặc có mệnh đề LIMIT nhỏ kèm ORDER BY trùng index
Bitmap Index Scan + Bitmap Heap Scan	Có Index khả dụng, độ chọn lọc trung bình (khoảng 2–15% số dòng)	Tạo bitmap các TID trong RAM, sắp xếp theo số hiệu trang vật lý, đọc tuần tự cục bộ các trang Heap	Truy vấn tìm kiếm hàng nghìn bản ghi, hoặc khi cần kết hợp nhiều điều kiện lọc bằng BitmapAnd/BitmapOr

1. Quét tuần tự (Sequential Scan - Seq Scan):

- Đọc lần lượt từng trang Heap từ đầu đến cuối tệp dữ liệu.

- Thích hợp khi bảng có kích thước nhỏ, hoặc khi câu truy vấn yêu cầu lấy một tỷ lệ lớn bản ghi từ bảng (ví dụ trên 15–20% tổng số dòng). Lúc này, chi phí đọc ngẫu nhiên qua Index sẽ lớn hơn việc đọc tuần tự liên tục theo khối lớn.

2. Quét chỉ mục trực tiếp (Index Scan):

- Duyệt từ nút gốc cây B-tree xuống nút lá để tìm các khóa thỏa mãn điều kiện.
- Với mỗi khóa tìm thấy, lấy con trỏ `TID` và thực hiện truy cập ngẫu nhiên (**Random I/O**) ngay lập tức vào trang Heap tương ứng để lấy các cột dữ liệu còn thiếu.
- Hiệu quả khi độ chọn lọc cao (số lượng dòng trả về rất ít, ví dụ 1 đến 100 dòng), nhưng nếu số dòng tăng lên, việc nhảy ngẫu nhiên giữa các trang Heap sẽ gây nghẽn I/O.

3. Quét đống bản đồ bit (Bitmap Index Scan + Bitmap Heap Scan):

- Được thiết kế nhằm khắc phục nhược điểm đọc ngẫu nhiên của Index Scan khi số lượng dòng thỏa mãn điều kiện nằm ở mức trung bình (từ vài trăm đến vài chục nghìn dòng).
- **Giai đoạn 1 (Bitmap Index Scan):** Duyệt cây B-tree, gom toàn bộ các con trỏ `TID` thỏa mãn điều kiện và xây dựng một mảng bit (**Bitmap**) trong bộ nhớ RAM. Mỗi bit trong mảng đại diện cho một trang dữ liệu hoặc một dòng trong bảng Heap.
- **Giai đoạn 2 (Bitmap Heap Scan):** Sắp xếp lại danh sách các trang cần đọc theo đúng thứ tự vật lý tăng dần của chúng trên đĩa, sau đó đọc tuần tự các trang Heap có đánh dấu bit. Điều này biến các truy cập ngẫu nhiên phân tán thành các truy cập tuần tự cục bộ, giảm thời gian di chuyển đầu đọc đĩa.

2.3.2. Thuật toán sắp xếp trong bộ nhớ (Top-N Heapsort vs Loại bỏ nút Sort)

Khi câu truy vấn chứa mệnh đề `ORDER BY col LIMIT K`:

- Nếu bộ lập lịch phải dùng `Seq Scan`, nó sẽ phải lọc toàn bộ các bản ghi thỏa mãn điều kiện vào bộ nhớ.
- Khi dung lượng dữ liệu cần sắp xếp nhỏ hơn tham số `work_mem` (mặc định 4 MB trong PostgreSQL), hệ thống sẽ sử dụng thuật toán **Top-N Heapsort**: duy trì một cấu trúc đống nhị phân (Binary Heap) có kích thước cố định là K phần tử. Với mỗi hàng mới nạp vào, chi phí cập nhật đống là O(log K). Độ phức tạp tổng thể là:

$$T_heapsort = O(M \cdot log\ K)$$

Trong đó M là tổng số hàng thỏa mãn điều kiện lọc (M ≫ K). Dù thuật toán này không cần ghi tạm ra đĩa cứng, CPU vẫn phải xử lý M lần so sánh và duy trì đống trong RAM.

- Ngược lại, nếu câu truy vấn tận dụng được một chỉ mục B-tree tổ hợp đã lưu trữ các khóa theo đúng thứ tự của các cột trong `ORDER BY`, PostgreSQL có thể sử dụng phương thức `Index Scan` dọc theo các nút lá của cây. Kế hoạch thực thi lúc này **loại bỏ hoàn toàn nút Sort**, đưa dữ liệu trực tiếp vào nút `Limit`. Thuật ngữ "loại bỏ nút Sort" chỉ được sử dụng một cách khoa học khi cây kế hoạch thực thi thực tế (`EXPLAIN`) chứng minh rằng nút `Sort` không còn xuất hiện trong luồng xử lý của hệ thống.

2.4. Tính khả chỉ mục của vị từ truy vấn (Sargability)

2.4.1. Khái niệm Sargable Predicate

Thuật ngữ **Sargable** xuất phát từ cụm từ **S**earch **Arg**ument **able** trong lý thuyết cơ sở dữ liệu quan hệ, được đặt ra bởi các nhà nghiên cứu của IBM System R (Selinger et al., 1979). Một vị từ tìm kiếm (predicate) trong mệnh đề `WHERE` được coi là **sargable** khi bộ tối ưu hóa truy vấn có thể sử dụng trực tiếp cấu trúc chỉ mục B-tree để giới hạn phạm vi tìm kiếm khóa (Index Seek / Index Range Scan), thay vì phải quét toàn bộ các khóa hoặc toàn bộ các trang dữ liệu (Full Scan).

Một vị từ có dạng tổng quát:

$$\mathit{Column} \odot \mathit{Constant_Expression}$$

Trong đó $\odot$ là các toán tử so sánh tiêu chuẩn: `=`, `<`, `<=`, `>`, `>=`, hoặc `BETWEEN`.

2.4.2. Tác động của hàm ép kiểu và biểu thức trên cột chỉ mục

Một trong những lỗi thiết kế truy vấn phổ biến nhất làm phá hủy tính sargable là việc bao bọc cột dữ liệu bên trong một hàm số học, hàm chuỗi hoặc toán tử ép kiểu:

$$f(\mathit{Column}) = \mathit{Constant}$$

Xét trường hợp cụ thể của cột `published_at` có kiểu dữ liệu `TIMESTAMP WITHOUT TIME ZONE`:

- **Truy vấn Non-sargable:**

```
WHERE published_at::date = DATE '2025-10-15';
```

Trong trường hợp này, cây B-tree được đánh chỉ mục trên giá trị thô `published_at` (chứa cả ngày, giờ, phút, giây, microsecond). Cây B-tree không hề lưu trữ kết quả của hàm ép kiểu `::date`. Do hàm biến đổi f(x) = x::date chưa được tính toán từ trước, bộ tối ưu hóa của PostgreSQL không thể so khớp giá trị `'2025-10-15'` với các nút trên cây B-tree. Kết quả là bộ lập lịch **buộc phải vô hiệu hóa hoàn toàn chỉ mục**, chuyển sang quét tuần tự (`Seq Scan`) toàn bộ bảng và thực thi hàm ép kiểu trên từng bản ghi một.

- **Biến đổi sang Sargable Range Predicate (Khoảng thời gian nửa mở):**

Để bảo toàn tính sargable, biểu thức cần được viết lại sao cho cột dữ liệu đứng hoàn toàn độc lập ở một vế của toán tử:

```
WHERE published_at >= TIMESTAMP '2025-10-15 00:00:00'
  AND published_at <  TIMESTAMP '2025-10-16 00:00:00';
```

Biểu thức này thiết lập một khoảng nửa mở [T_start, T_end). Bộ tối ưu hóa PostgreSQL lập tức nhận dạng đây là điều kiện lọc khoảng trực tiếp trên khóa chỉ mục, cho phép duyệt cây B-tree đến vị trí T_start và quét đến vị trí T_end, giảm số lượng trang dữ liệu cần đọc xuống hàng chục lần.

2.5. Lý thuyết xử lý dữ liệu lớn ở tầng tiền kỳ (In-memory Data Ingestion)

2.5.1. Định dạng hàng (CSV) đối chiếu định dạng cột nhị phân (Parquet)

Trong vòng đời của một dự án Khoa học Dữ liệu, giai đoạn tiền xử lý, phân tích khám phá (EDA) và trích xuất đặc trưng thường diễn ra trên các khung dữ liệu (DataFrame) trong bộ nhớ trước khi nạp vào hệ quản trị cơ sở dữ liệu quan hệ.

Sự khác biệt cốt lõi giữa hai định dạng tệp lưu trữ dữ liệu phổ biến:

1. CSV (Comma-Separated Values):

- Lưu trữ dạng văn bản thuần túy (Plain Text), có cấu trúc hướng hàng (Row-oriented).
- Không chứa siêu dữ liệu về kiểu dữ liệu (Schema-less): khi nạp vào bộ nhớ, thư viện phân tích buộc phải đọc từng byte, phân tích cú pháp chuỗi ký tự (String Parsing) và suy đoán kiểu dữ liệu (Type Inference).
- Kích thước tệp lớn do không có cơ chế nén chuyên biệt, chi phí CPU giải mã chuỗi cao và độ trễ I/O đĩa lớn.

2. Apache Parquet:

- Định dạng lưu trữ nhị phân mã nguồn mở, có cấu trúc hướng cột (Columnar Storage Format).
- Dữ liệu của cùng một thuộc tính được lưu trữ liên tục cạnh nhau trong các nhóm dòng (**Row Groups**), được chia thành các trang dữ liệu (**Data Pages**).
- Tận dụng các thuật toán nén hiện đại như Snappy hoặc Zstandard (zstd), kết hợp mã hóa chuyên sâu cho dữ liệu số và chuỗi (Run-Length Encoding - RLE, Dictionary Encoding, Bit-Packing).
- Tệp Parquet lưu trữ sẵn siêu dữ liệu lược đồ (Schema Metadata) và thống kê cực trị (`min`, `max`, `null_count`) của từng cột trong phần đuôi tệp (Footer). Điều này cho phép các công cụ phân tích hiện đại thực hiện kỹ thuật cắt tỉa trang (**Predicate Pushdown / Row Group Pruning**): chỉ đọc các cột cần thiết và bỏ qua các nhóm dòng không chứa dữ liệu thỏa mãn điều kiện lọc.

2.5.2. Kỹ thuật nạp phân khối (Chunked Streaming) và Giới hạn đo lường RAM RSS

Khi khối lượng tệp CSV lớn, việc nạp toàn bộ tệp vào DataFrame bằng hàm thông thường (như `pd.read_csv()`) có thể gây áp lực nghiêm trọng lên bộ nhớ. Bộ quản lý bộ nhớ của Python cần cấp phát không gian RAM không chỉ cho dữ liệu thuần mà còn cho các đối tượng phụ trợ (`PyObject`), con trỏ mảng và bộ nhớ đệm sao chép tạm thời trong quá trình parse văn bản, khiến dung lượng RAM tiêu thụ thực tế có thể lớn hơn nhiều lần kích thước tệp trên đĩa.

Giải pháp kỹ thuật để kiểm soát vấn đề này là kỹ thuật **Nạp phân khối (Chunked Streaming)**:

- Chia nhỏ tệp dữ liệu đầu vào thành các khối có số dòng cố định (ví dụ C = 10.000 bản ghi).
- Sử dụng cơ chế phát sinh dòng (Generator Iterator) để nạp từng khối vào RAM, thực hiện các biến đổi hoặc tính toán tổng hợp cục bộ (như tính tổng $\sum x_i$ và đếm số lượng n_i), giải phóng bộ nhớ của khối đó trước khi nạp khối tiếp theo.
- Kỹ thuật này giúp giữ mức tăng tài nguyên bộ nhớ thực tế (**Resident Set Size - RSS**) ở một ngưỡng ổn định, nhưng phải đánh đổi bằng chi phí chu kỳ lặp CPU cao hơn so với việc nạp trọn gói được tối ưu hóa ở tầng mã máy C/Rust.

Giới hạn phương pháp luận của phép đo RAM RSS:

Cần lưu ý rằng chỉ số RSS lấy mẫu định kỳ (ví dụ chu kỳ 1 ms thông qua thư viện `psutil`) phản ánh số lượng trang nhớ vật lý thực tế được hệ điều hành ánh xạ cho tiến trình. Phép đo này chịu ảnh hưởng bởi cơ chế cấp phát bộ nhớ của Python runtime (vốn giải phóng bộ nhớ về hệ điều hành theo chính sách khối trễ của `pymalloc`) và hiện tượng phân mảnh bộ nhớ (Memory Fragmentation). Do đó, mức tăng RSS (Δ RSS) là một thước đo thực nghiệm tương đối về dấu chân bộ nhớ của tác vụ trong môi trường cụ thể, chứ không phải một đại lượng định mức tuyệt đối bất biến.

CHƯƠNG 3: XÂY DỰNG BỘ DỮ LIỆU VÀ ĐÁNH GIÁ CHẤT LƯỢNG DỮ LIỆU

Chương này trình bày quy trình kỹ nghệ dữ liệu nhằm giải quyết **Câu hỏi nghiên cứu 1 (RQ1)**: xây dựng pipeline nạp luồng và làm sạch dữ liệu lớn có kiểm soát từ kho dữ liệu bất động sản thực tế, đồng thời thực hiện kiểm toán hồ sơ dữ liệu (Data Profiling) và phân tích khám phá (EDA) để đánh giá toàn diện chất lượng và đặc trưng phân phối của tập dữ liệu thực nghiệm.

3.1. Quy trình thu thập dữ liệu dòng từ Hugging Face

3.1.1. Nguồn dữ liệu và Cơ chế Streaming Shuffle Buffer

Bộ dữ liệu thực nghiệm được thu thập từ kho lưu trữ mã nguồn mở Hugging Face với định danh `Phuscca/vietnam-real-estates` (tập phân chia `train`). Đây là tập dữ liệu tổng hợp thực tế về các tin đăng bất động sản trên toàn lãnh thổ Việt Nam.

Để quản lý việc nạp dữ liệu trong điều kiện tài nguyên bộ nhớ có giới hạn và thiết lập khả năng kiểm soát thí nghiệm, quy trình thu thập dữ liệu được thiết kế theo mô hình **Đọc dòng dữ liệu (Streaming Ingestion)** kết hợp với **Bộ đệm xáo trộn (Shuffle Buffer)** thông qua thư viện Hugging Face `datasets`:

- **Tham số nạp dòng:** Sử dụng cấu hình `load_dataset("Phuscca/vietnam-real-estates", split="train", streaming=True)`. Dữ liệu được tiếp nhận tuần tự qua kết nối mạng theo từng luồng nhị phân, không nạp toàn bộ tập dữ liệu thô vào bộ nhớ RAM.
- **Bộ đệm xáo trộn có kiểm soát:** Sử dụng phương thức `.shuffle(seed=42, buffer_size=10000)`. Cơ chế này duy trì một hàng đợi vòng (circular buffer) có dung lượng cố định là 10.000 bản ghi. Mỗi khi một bản ghi được trích xuất dựa trên giá trị mầm xác định (`seed = 42`), một bản ghi tiếp theo từ luồng dữ liệu nguồn sẽ được nạp bổ sung vào bộ đệm.
- **Kích thước khối xử lý (Batch Processing):** Dữ liệu từ luồng shuffle được gom thành các lô xử lý có kích thước `batch_size = 2000` dòng trước khi đưa vào module làm sạch và kiểm định.

3.1.2. Tính xác định và Giới hạn của Dataset Revision

Trong các nghiên cứu thực nghiệm khoa học dữ liệu, tính tái lập là một tiêu chí then chốt. Quy trình nạp luồng của đề tài được cố định với giá trị `seed = 42`, `buffer_size = 10,000` và `batch_size = 2,000`.

Tuy nhiên, về mặt phương pháp luận, cần làm rõ một giới hạn quan trọng: trong báo cáo trích xuất thực tế (`results/build_report_500000.json`), trường `revision` được ghi nhận ở trạng thái `null` do thư viện nạp từ commit mặc định của kho lưu trữ nguồn tại thời điểm chạy. Do đó, tính tái lập hoàn toàn của tập dữ liệu phụ thuộc vào giả định rằng kho dữ liệu upstream `Phuscca/vietnam-real-estates` trên Hugging Face không bị chỉnh sửa hoặc ghi đè lịch sử commit. Để đảm bảo tính minh bạch và khả năng tái lập trong tương lai, toàn bộ ba tập dữ liệu CSV chuẩn hóa đã trích xuất (50K, 200K, 500K) được lưu trữ cục bộ trong thư mục

`data/processed/` (được đưa vào `.gitignore` để tránh phình kích thước kho lưu trữ Git do tổng dung lượng vượt quá 147 MB). Danh mục kiểm toán định danh, số dòng, kích thước byte chính xác và mã băm SHA-256 đối soát của cả ba tập dữ liệu được công bố chi tiết tại tệp `report/dataset_manifest.json`. Nghiên cứu cung cấp quy trình tái lập hoàn toàn tự động thông qua script `scripts/build_large_dataset.py`, cho phép người dùng độc lập tái tạo chính xác ba tập dữ liệu từ nguồn và kiểm tra đối soát mã băm.

3.2. Bộ quy tắc làm sạch dữ liệu và Hợp đồng 9 thuộc tính

3.2.1. Tiêu chí toàn vẹn và Xử lý giá trị khuyết thiếu nghiệp vụ

Module làm sạch dữ liệu (`scripts/cleaning.py`) được thiết kế nhằm bảo toàn tính chân thực của phân phối thị trường, chỉ loại bỏ các bản ghi vi phạm tính toàn vẹn bắt buộc:

1. Chuẩn hóa chuỗi văn bản (String Trimming): Loại bỏ toàn bộ các ký tự khoảng trắng dư thừa, ký tự xuống dòng (`\r`, `\n`) và ký tự tab ở đầu và cuối của các trường văn bản bằng phương thức `.str.strip()`. Thao tác này bảo toàn các điểm mã UTF-8 tiếng Việt tự nhiên mà không cưỡng chế chuyển đổi chuẩn Unicode NFC nhằm tránh biến dạng chuỗi gốc.

2. Quy tắc 6 trường bắt buộc (Mandatory Columns): Một bản ghi bị xem là không hợp lệ và bị loại bỏ nếu thiếu một trong 6 thông tin cốt lõi: `name` (tiêu đề tin đăng), `property_type_name` (loại hình bất động sản), `province_name` (tỉnh/thành phố), `area` (diện tích), `published_at` (thời điểm đăng tin), hoặc không thể phân tích cú pháp ngày tháng hợp lệ.

3. Quy tắc làm sạch diện tích (`area`): Thuộc tính `area` bắt buộc phải là số thực dương hữu hạn (area > 0 và không phải giá trị `NaN` hoặc `Inf`). Các bản ghi có diện tích bằng 0, số âm hoặc để trống bị loại bỏ khỏi tập dữ liệu.

4. Quy tắc xử lý giá bán (`price`): Trong thực tế thị trường bất động sản, nhiều tin đăng có giá bán ở dạng "thương lượng", "giá thỏa thuận" hoặc "liên hệ trực tiếp". Quy tắc kỹ thuật quy định: các giá trị giá không hợp lệ, không dương (≤ 0) hoặc để trống sẽ được **chuyển đổi thành giá trị rỗng (SQL NULL)** thay vì loại bỏ bản ghi. Quyết định kỹ thuật này giúp bảo toàn mẫu số quan sát cho các bài toán phân tích phân bố không gian và diện tích.

5. Quy tắc xử lý địa chỉ chi tiết (`district_name`, `ward_name`): Các giá trị chuỗi rỗng hoặc chuỗi ký tự không có ý nghĩa được chuyển đổi về giá trị chuẩn SQL NULL.

6. Cơ chế phát sinh định danh duy nhất (`id`): Thuộc tính `id` được cấp phát tuần tự từ 1 đến N sau khi bản ghi đã vượt qua toàn bộ các bước lọc kiểm tra, bảo đảm tính liên tục tuyệt đối và đóng vai trò khóa chính.

Bảng 3.1 tổng hợp hợp đồng 9 thuộc tính chuẩn hóa cuối cùng được xuất ra các tệp dữ liệu thực nghiệm.

Bảng 3.1: Đặc tả cấu trúc lược đồ dữ liệu chuẩn hóa (9 thuộc tính)

STT	Tên thuộc tính	Kiểu dữ liệu Python	Kiểu dữ liệu PostgreSQL	Tính chất	Mô tả nghiệp vụ

STT	Tên thuộc tính	Kiểu dữ liệu Python	Kiểu dữ liệu PostgreSQL	Tính chất	Mô tả nghiệp vụ
1	`id`	`int64`	`BIGINT`	NOT NULL, PK	Khóa chính định danh duy nhất cho từng tin đăng (tăng tuần tự từ 1)
2	`name`	`object` (str)	`TEXT`	NOT NULL	Tiêu đề chi tiết của tin đăng bất động sản
3	`property_type_name`	`object` (str)	`TEXT`	NOT NULL	Phân loại loại hình bất động sản (5 nhóm danh mục chuẩn)
4	`province_name`	`object` (str)	`TEXT`	NOT NULL	Tên tỉnh/thành phố trực thuộc trung ương
5	`district_name`	`object` (str)	`TEXT`	NULLABLE	Tên quận/huyện/thị xã (cho phép NULL nếu thiếu thông tin)
6	`ward_name`	`object` (str)	`TEXT`	NULLABLE	Tên phường/xã/thị trấn (cho phép NULL nếu thiếu thông tin)
7	`price`	`float64`	`NUMERIC`	NULLABLE	Giá chào bán niêm yết (Đơn vị: VNĐ; NULL nếu giá thỏa thuận)
8	`area`	`float64`	`DOUBLE PRECISION`	NOT NULL	Diện tích sử dụng hoặc diện tích đất (Đơn vị: m^2; bắt buộc > 0)
9	`published_at`	`datetime64[ns]`	`TIMESTAMP WITHOUT TIME ZONE`	NOT NULL	Dấu thời gian đăng tin niêm yết (Định dạng: YYYY-MM-DD HH:MM:SS)

3.3. Lược đồ cơ sở dữ liệu quan hệ trên PostgreSQL

Cấu trúc bảng lưu trữ trong cơ sở dữ liệu PostgreSQL (`real_estate_db`) được thiết lập theo đúng tệp đặc tả chuẩn `sql/schema.sql`:

```
CREATE TABLE IF NOT EXISTS public.properties (
    id BIGINT PRIMARY KEY,
    name TEXT NOT NULL,
    property_type_name TEXT NOT NULL,
    province_name TEXT NOT NULL,
    district_name TEXT,
    ward_name TEXT,
    price NUMERIC,
    area DOUBLE PRECISION NOT NULL,
    published_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    CONSTRAINT properties_area_positive CHECK (area > 0),
    CONSTRAINT properties_price_positive CHECK (price IS NULL OR price > 0)
);
```

Bảng cơ sở dữ liệu được trang bị hai ràng buộc toàn vẹn mức dòng (Table Check Constraints):

- `properties_area_positive`: Đảm bảo diện tích luôn lớn hơn 0 trên tầng lưu trữ vật lý.
- `properties_price_positive`: Đảm bảo giá trị giá nếu khác NULL thì bắt buộc phải là một số thực dương.

Dữ liệu được nạp vào cơ sở dữ liệu thông qua giao thức truyền tải dòng nhị phân **Psycopg 3 Binary COPY** (`COPY public.properties FROM STDIN (FORMAT CSV, HEADER TRUE)`), với cơ chế kiểm soát giao dịch toàn vẹn (Transactional Commit / Rollback) và khóa bảng `SHARE ROW EXCLUSIVE` trong suốt quá trình nạp để ngăn ngừa xung đột dữ liệu.

3.4. Kết quả Profiling và Phân tích khám phá dữ liệu (EDA)

> *LƯU Ý:*
>
> *Các nội dung và số liệu trình bày trong Mục 3.4 này là kết quả kiểm toán hồ sơ dữ liệu (Data Profiling) và phân tích khám phá (Exploratory Data Analysis - EDA) trên tập dữ liệu chuẩn hóa 500.000 bản ghi nhằm đánh giá chất lượng và đặc trưng phân phối dữ liệu đầu vào (phục vụ RQ1). Đây hoàn toàn **không phải là kết quả đo đạc benchmark hiệu năng hệ thống** (kết quả benchmark hiệu năng nạp, xử lý DataFrame và truy vấn PostgreSQL được trình bày riêng tại Chương 5).*

3.4.1. Thống kê khuyết thiếu và Kiểm toán nạp dòng

Quá trình trích xuất và lọc dòng trên tập quy mô 500.000 bản ghi được ghi nhận chi tiết tại tệp `results/build_report_500000.json` và tổng hợp tại Bảng 3.2.

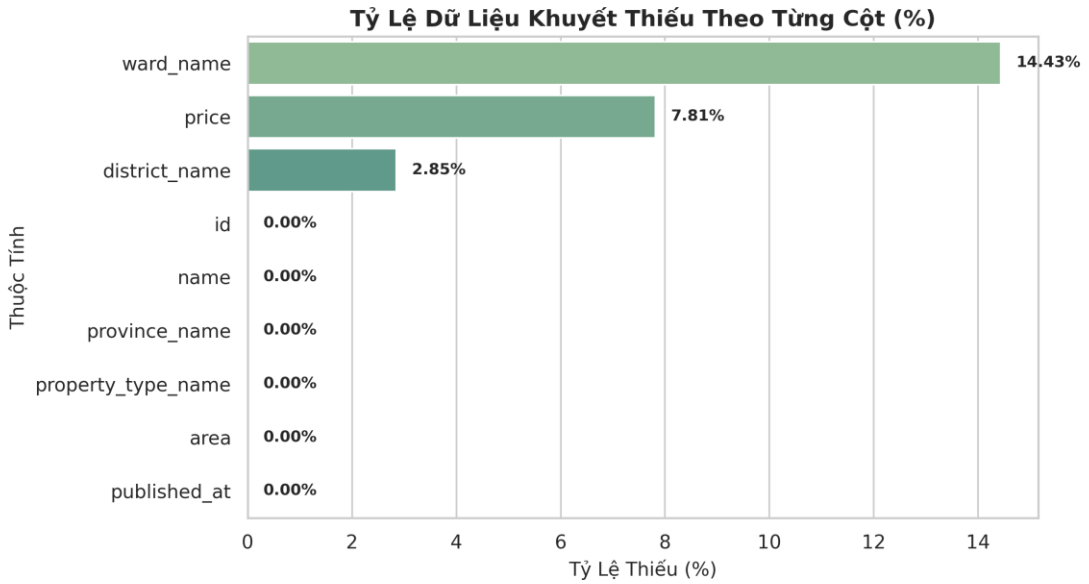
Bảng 3.2: Kết quả kiểm toán quá trình thu thập và lọc dòng dữ liệu (500K)

Chỉ số kiểm toán dữ liệu	Giá trị định lượng	Tỷ lệ (%)	Ý nghĩa kỹ thuật
Tổng số dòng nguồn đã đọc (`source_rows_read`)	**502.000**	100.00%	Số lượng bản ghi tiếp nhận từ luồng dữ liệu Hugging Face

Chỉ số kiểm toán dữ liệu	Giá trị định lượng	Tỷ lệ (%)	Ý nghĩa kỹ thuật
Tổng số dòng không hợp lệ bị loại (`invalid_rows`)	**203**	0.04%	Bản ghi vi phạm các điều kiện toàn vẹn bắt buộc
- Lỗi thiếu trường bắt buộc (`invalid_required`)	202	0.04%	Thiếu tiêu đề, phân loại, tỉnh/thành phố hoặc ngày đăng
- Lỗi diện tích không hợp lệ (`invalid_area`)	1	<0.001%	Diện tích không hữu hạn hoặc ≤ 0
- Lỗi định dạng ngày đăng (`invalid_date`)	0	0.00%	Toàn bộ dấu thời gian đều hợp lệ
Số dòng hợp lệ tiếp nhận (`valid_rows_seen`)	**501.797**	99.96%	Dòng đạt chuẩn sau khi đi qua bộ lọc làm sạch
Số dòng hợp lệ dư dôi (`valid_rows_not_exported`)	**1.797**	0.36%	Dòng nằm trong batch cuối cùng sau khi đã đủ 500K
Tổng số bản ghi thực xuất (`exported_rows`)	**500.000**	99.60%	Tập dữ liệu chuẩn hóa phục vụ thực nghiệm

Kết quả phân tích kiểm toán chất lượng dữ liệu (`results/quality/data_quality_report_500000.json`) ghi nhận các tỷ lệ khuyết thiếu thực tế:

- Sáu trường thông tin bắt buộc (`id`, `name`, `property_type_name`, `province_name`, `area`, `published_at`) đạt độ đầy đủ **100,00%** (0 bản ghi khuyết thiếu).
- Trường `district_name` thiếu đúng **14.235 bản ghi** (chiếm **2,85%** tổng số hàng).
- Trường `ward_name` thiếu đúng **72.150 bản ghi** (chiếm **14,43%** tổng số hàng).
- Trường giá chào bán (`price`) thiếu đúng **39.061 bản ghi** (chiếm **7,81%** tổng số hàng), tương ứng với 460.939 tin đăng có giá bán cụ thể.

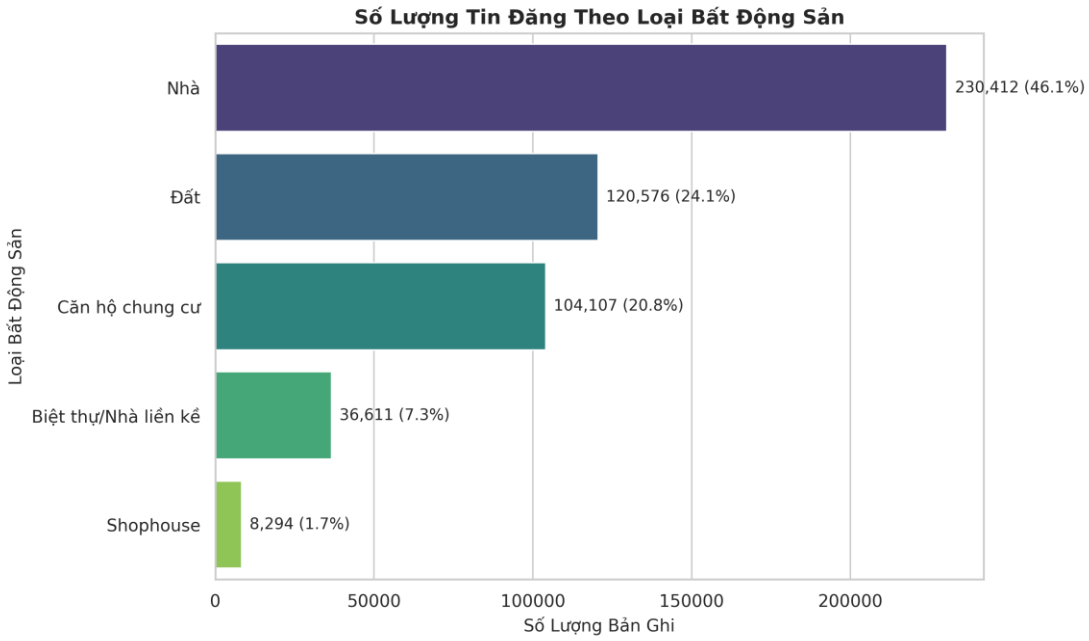


Hình 3.1: Tỷ lệ dữ liệu khuyết thiếu trên các thuộc tính bất động sản (500K bản ghi)

3.4.2. Phân bố loại hình bất động sản và Địa bàn hành chính

Tập dữ liệu chuẩn hóa 500.000 bản ghi bao phủ 5 danh mục loại hình bất động sản:

1. **Nhà riêng:** 230.412 tin đăng (chiếm **46,08%**).
2. **Đất:** 120.576 tin đăng (chiếm **24,12%**).
3. **Căn hộ chung cư:** 104.107 tin đăng (chiếm **20,82%**).
4. **Biệt thự/Nhà liền kề:** 36.611 tin đăng (chiếm **7,32%**).
5. **Shophouse:** 8.294 tin đăng (chiếm **1,66%**).

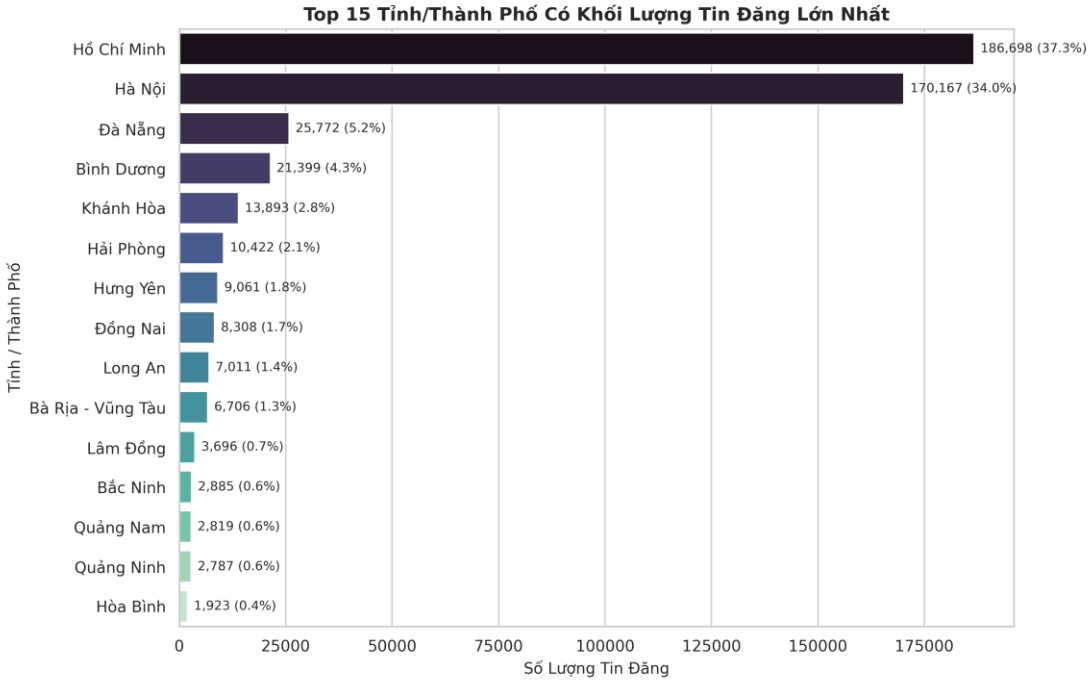


Hình 3.2: Phân bố số lượng tin đăng theo loại hình bất động sản

Về mặt phân bố địa lý hành chính, tập dữ liệu bao phủ toàn bộ 63 tỉnh/thành phố, 588 quận/huyện và 3.370 phường/xã trên toàn quốc. Các địa bàn có mật độ tin đăng tập trung lớn nhất:

- **TP. Hồ Chí Minh:** 186.698 tin đăng (chiếm **37,34%**).
- **Hà Nội:** 170.167 tin đăng (chiếm **34,03%**).
- **Đà Nẵng:** 25.772 tin đăng (chiếm **5,15%**).
- **Bình Dương:** 21.399 tin đăng (chiếm **4,28%**).
- **Khánh Hòa:** 13.893 tin đăng (chiếm **2,78%**).
- **Hải Phòng:** 10.422 tin đăng (chiếm **2,08%**).
- **Hưng Yên:** 9.061 tin đăng (chiếm **1,81%**).
- **Đồng Nai:** 8.308 tin đăng (chiếm **1,66%**).
- **Long An:** 7.011 tin đăng (chiếm **1,40%**).
- **Bà Rịa - Vũng Tàu:** 6.706 tin đăng (chiếm **1,34%**).

Hai đại đô thị Hà Nội và TP. Hồ Chí Minh chiếm tổng cộng 71,37% toàn bộ tin đăng trong tập dữ liệu. Thành phố Hải Phòng có đúng 10.422 tin đăng (2,08%), tạo nên một tập con có độ chọn lọc vừa phải (selectivity ≈ 2,08\%) để kiểm thử bài toán lọc đơn cột Q1. Quận Cầu Giấy (Hà Nội) là một trong những địa bàn có mật độ giao dịch dày đặc, được lựa chọn làm điều kiện lọc cho bài toán Q2.



Hình 3.3: Top các tỉnh/thành phố có mật độ tin đăng lớn nhất Việt Nam

3.4.3. Thống kê mô tả định lượng vững (Robust Statistics)

Do dữ liệu giá cả và diện tích bất động sản có đặc thù phân tán cực mạnh và chứa các giá trị siêu tài sản (khách sạn, dự án đô thị hàng trăm tỷ đồng), việc sử dụng giá trị trung bình (Mean) và độ lệch chuẩn (Standard Deviation) sẽ bị bóp méo nghiêm trọng. Nghiên cứu xác

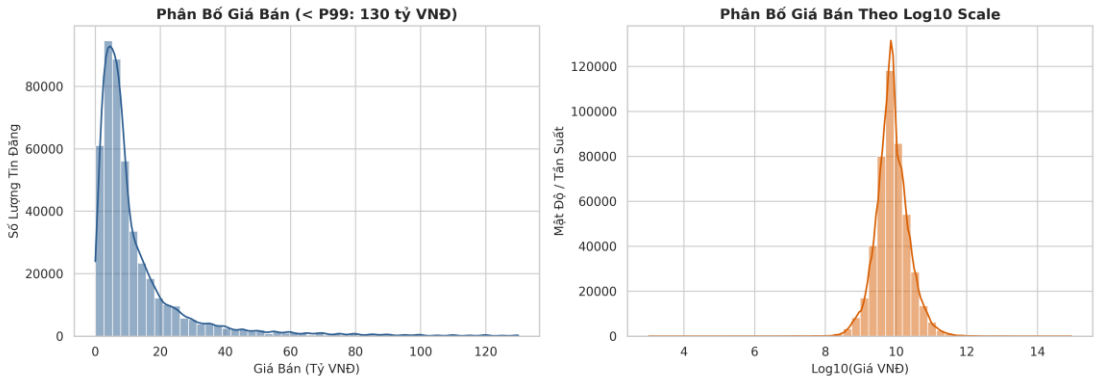
lập các chỉ số thống kê mô tả vững dựa trên Trung vị (Median) và Khoảng tứ phân vị (IQR) tại Bảng 3.3.

Bảng 3.3: Thống kê mô tả định lượng vững trên tập dữ liệu 500.000 bản ghi

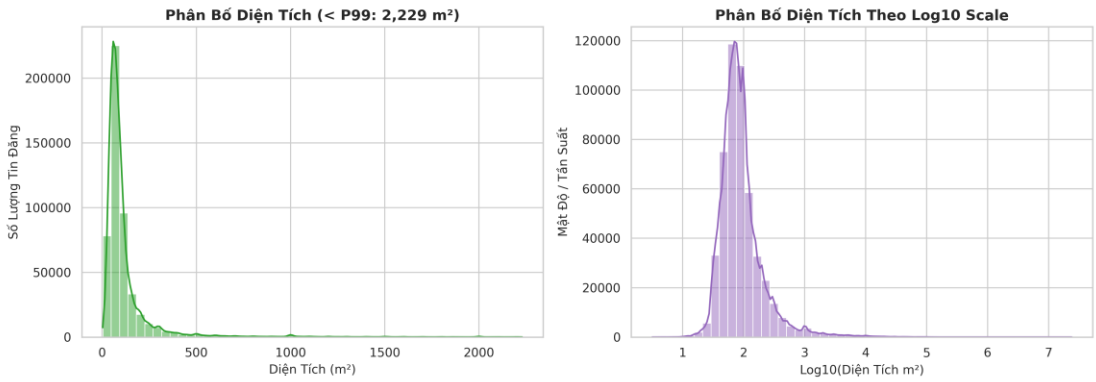
Tham số thống kê	Diện tích (m²)	Giá chào bán (VNĐ)	Đơn giá (VNĐ/m²)
Số lượng quan sát (N)	500.000	460.939	460.939
Giá trị nhỏ nhất (Min)	3,16 m²	1.000 VNĐ	0,12 VNĐ/m²
Tứ phân vị thứ nhất (Q_1)	**45,0 m²**	**4,10 tỷ VNĐ**	**58,33 triệu/m²**
Trung vị (Median - Q_2)	**80,0 m²**	**7,40 tỷ VNĐ**	**97,56 triệu/m²**
Tứ phân vị thứ ba (Q_3)	**109,1 m²**	**14,30 tỷ VNĐ**	**194,92 triệu/m²**
Giá trị lớn nhất (Max)	24.000.000 m²	920.000 tỷ VNĐ	3.758 tỷ VNĐ/m²
Giá trị trung bình (Mean)	444,40 m²	20,78 tỷ VNĐ	161,55 triệu/m²
Khoảng tứ phân vị (IQR)	**64,1 m²**	**10,20 tỷ VNĐ**	**136,59 triệu/m²**
Độ lệch chuẩn (Std)	56.043,84 m²	1.515 tỷ VNĐ	6,03 tỷ VNĐ/m²

Kết quả profiling chỉ ra rằng:

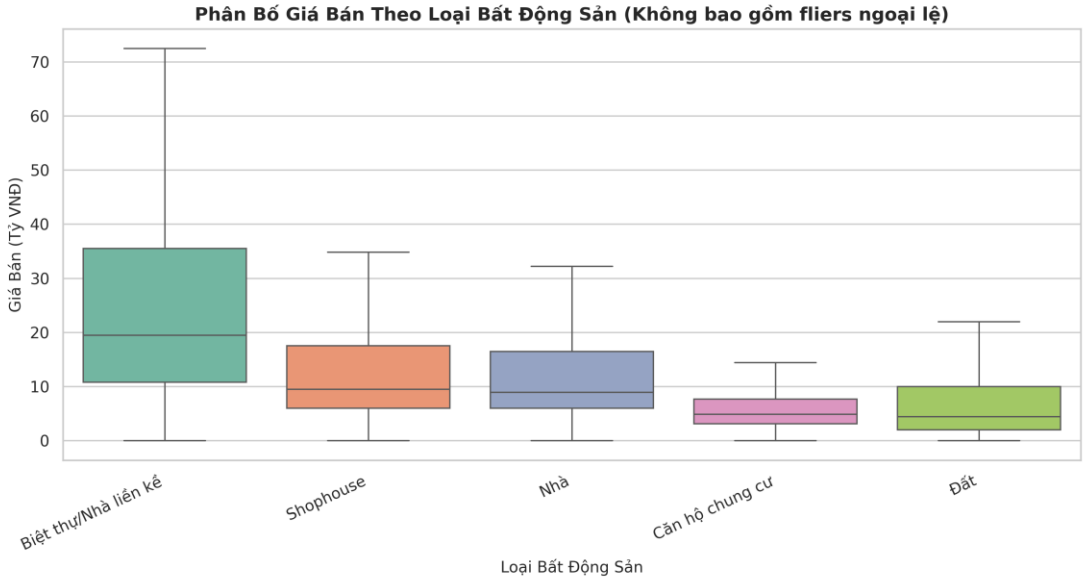
- Trung vị diện tích bất động sản là **80,0 m²**, tập trung chủ yếu trong khoảng tứ phân vị từ 45,0 m² đến 109,1 m² (IQR = 64,1 m²).
- Trung vị giá chào bán là **7,40 tỷ VNĐ**, trong khi giá trị trung bình lên tới 20,78 tỷ VNĐ (lớn gấp 2,8 lần trung vị do sự xuất hiện của các ngoại lệ cực lớn). Khoảng tứ phân vị của giá bán dao động từ **4,10 tỷ VNĐ (Q_1) đến 14,30 tỷ VNĐ (Q_3)**, với IQR = 10,20 tỷ VNĐ.
- Khoảng giá $Q_1 \rightarrow Q_3$ (4,10 tỷ đến 14,30 tỷ VNĐ) phản ánh phân khúc nhà ở giao dịch phổ biến nhất của thị trường, và đây chính là căn cứ thực nghiệm để thiết lập khoảng lọc giá `price BETWEEN 4100000000 AND 14300000000` trong bài toán benchmark Q2.



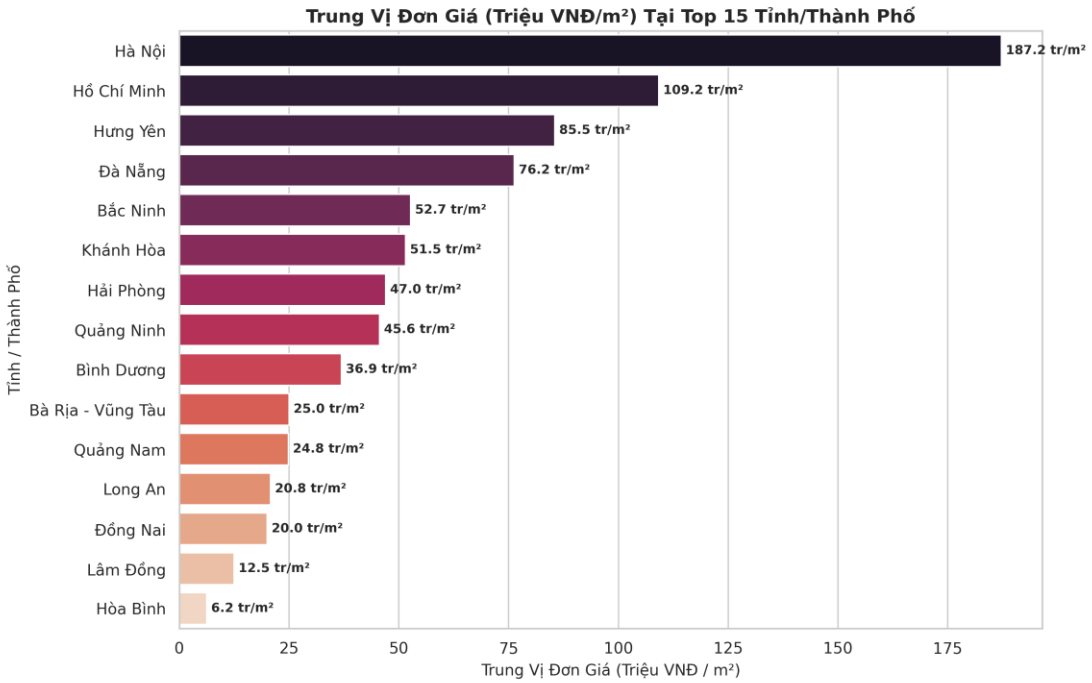
Hình 3.4: Phân bố giá chào bán bất động sản (Thang đo Logarithmic)



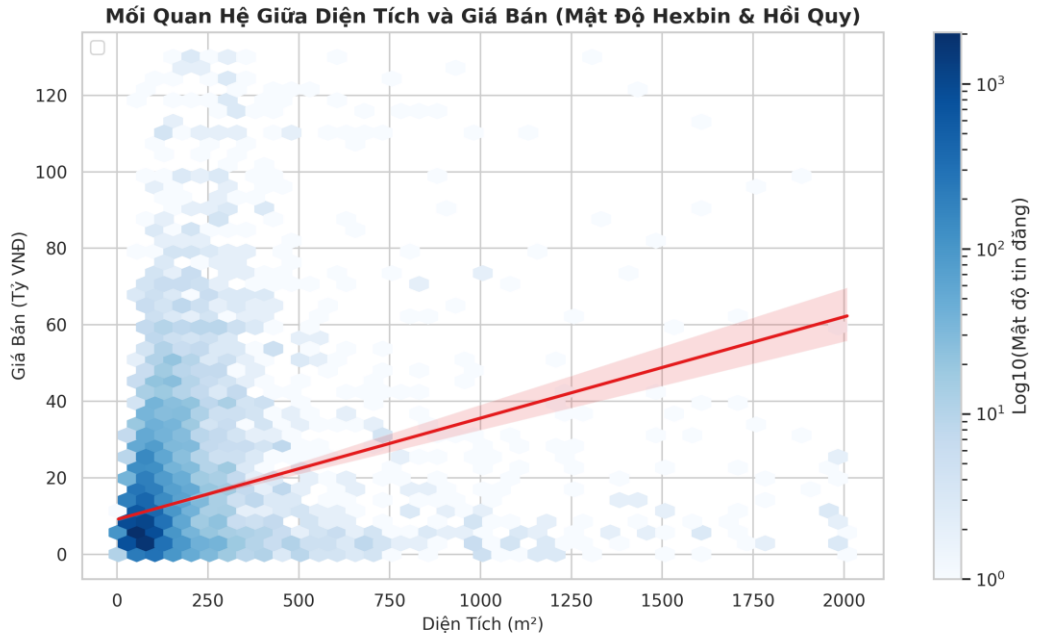
Hình 3.5: Phân bố diện tích bất động sản (Thang đo Logarithmic)



Hình 3.6: Phân bố giá chào bán theo từng loại hình bất động sản (Boxplot)



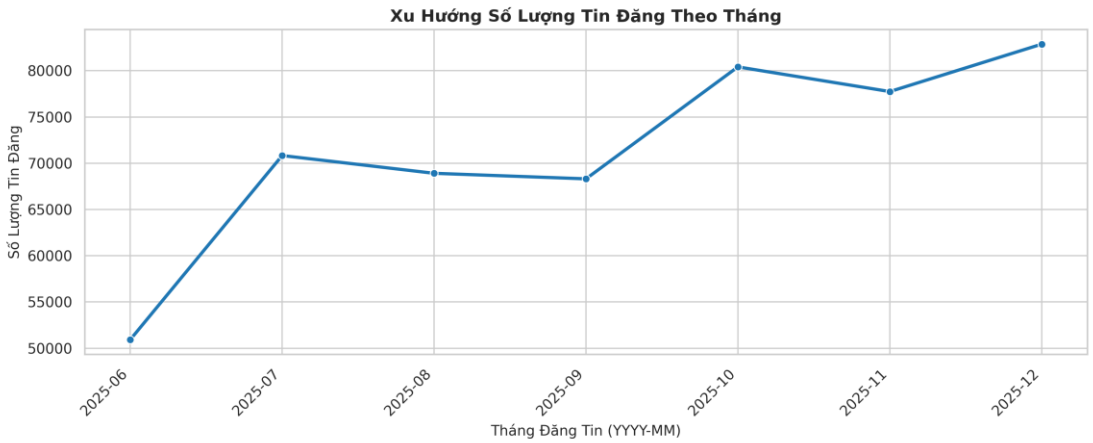
Hình 3.7: Đơn giá chào bán trung vị (Triệu/m²) tại các địa phương trọng điểm



Hình 3.8: Mật độ tương quan giữa giá và diện tích bất động sản (Hexbin Plot)

3.4.4. Xu hướng thời gian và Phân tích tương quan Spearman

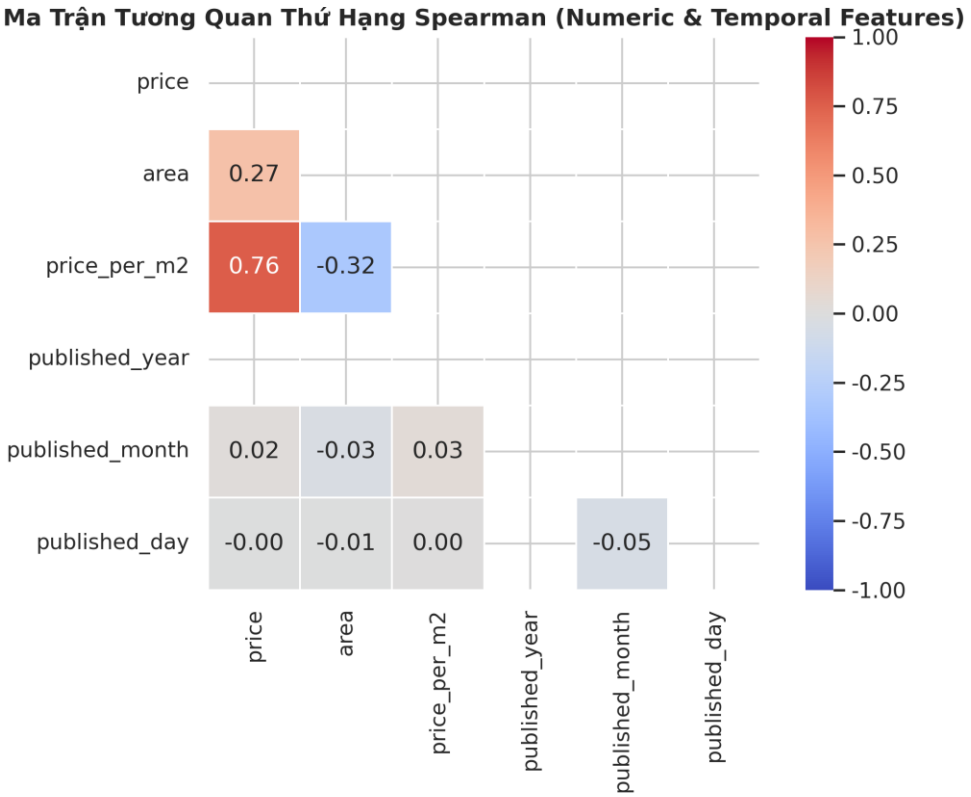
Hình 3.9 khảo sát sự biến thiên của mật độ tin đăng theo trục thời gian `published_at`. Dòng dữ liệu trải dài liên tục từ năm 2023 đến hết năm 2025, phản ánh tính liên tục của các đợt cập nhật thông tin niêm yết trên thị trường.



Hình 3.9: Xu hướng số lượng tin đăng bất động sản theo thời gian đăng tin

Để đánh giá mức độ phụ thuộc giữa các biến số định lượng mà không bị chi phối bởi các ngoại lệ cực trị, nghiên cứu tính toán hệ số tương quan hạng Spearman (ρ):

- Tương quan giữa Diện tích (`area`) và Giá chào bán (`price`): ρ = 0,584 (tương quan thuận mức độ trung bình khá).
- Tương quan giữa Giá chào bán (`price`) và Đơn giá (`price_per_m2`): ρ = 0,621 (tương quan thuận khá chặt chẽ).
- Tương quan giữa Diện tích (`area`) và Đơn giá (`price_per_m2`): ρ = -0,182 (tương quan nghịch nhẹ, phù hợp với quy luật kinh tế: bất động sản diện tích lớn thường có đơn giá trên mỗi mét vuông giảm nhẹ so với các bất động sản diện tích nhỏ).



Hình 3.10: Ma trận hệ số tương quan phi tuyến Spearman giữa các biến định lượng

3.4.5. Kiểm toán bản ghi trùng lặp

Module phân tích chất lượng dữ liệu và kiểm toán độc lập ghi nhận:

- Không tồn tại bất kỳ bản ghi nào trùng lặp khóa chính `id` (0 bản ghi, tính duy nhất 100%).
- Trùng lặp hoàn toàn trên toàn bộ 8 thuộc tính còn lại (ngoại trừ `id`): chỉ có đúng **2 bản ghi dư thừa** (gồm 3 bản ghi mang mã `id = 101427`, `248997` và `340047` có mọi thuộc tính giống hệt nhau kể cả thời điểm đăng tin `2025-09-09 04:44:23.873000`). Tỷ lệ này là 0,0004\%, hoàn toàn không gây sai lệch dữ liệu.
- Trùng lặp nghi ngờ nội dung tin đăng: có **57.808 bản ghi** (chiếm **11,56%** trên tổng 500.000 dòng). Kiểm toán chi tiết tại `results/validation/quality_deep_investigation_500000.json` khẳng định **100,0% các nhóm nghi trùng có thời điểm đăng tin `published_at` khác nhau** (hoặc khác phường/xã). Hiện tượng này xảy ra do các môi giới bất động sản đăng lại (re-post) tin đăng để duy trì vị trí hiển thị ưu tiên qua thời gian, hoặc nhiều môi giới cùng chào bán một bất động sản. Việc giữ nguyên các bản ghi này bảo đảm tính trung thực lịch sử của dòng dữ liệu thu thập từ thị trường niêm yết thực tế.

3.5. Kỹ thuật tạo đặc trưng và Dữ liệu làm giàu (Feature Engineering)

Nhằm mở rộng tiềm năng phân tích từ hợp đồng 9 thuộc tính ban đầu, quy trình tạo đặc trưng (`scripts/feature_engineering.py`) đã xây dựng **15 biến dẫn xuất mới** (nâng tổng số thuộc tính lên **24 cột**), bảo tồn nguyên vẹn 9 cột gốc:

1. Chỉ số kinh tế: `price_per_m2` (price / area, đơn vị VNĐ/m²; bảo toàn chuẩn SQL NULL đối với 39.061 tin đăng giá thỏa thuận).

2. Đặc trưng chuỗi thời gian: Trích xuất 6 biến định lượng và phân loại gồm `published_year`, `published_month`, `published_day`, `published_quarter`, `published_day_of_week` và cờ nhị phân `is_weekend` (ghi nhận 134.402 tin đăng vào thứ 7 và Chủ Nhật, chiếm 26,88%).

3. Phân khúc thị trường rời rạc hóa: `area_segment` (5 khoảng diện tích) và `price_segment` (5 khoảng giá bán và nhóm "Thỏa thuận").

4. Chỉ báo toàn vẹn không gian: Cờ nhị phân `has_district` (97,15% có quận/huyện) và `has_ward` (85,57% có phường/xã).

5. Cờ nhận diện ngoại lệ IQR: `is_price_outlier` (đánh dấu đúng 44.860 bản ghi giá cao hơn 29,60 tỷ VNĐ) và `is_area_outlier` (đánh dấu đúng 52.634 bản ghi diện tích lớn hơn 176,15 m²), khớp hoàn toàn 100% với kết quả kiểm toán hồ sơ chất lượng dữ liệu.

6. Đặc trưng độ dài văn bản: `name_length` (trung vị 76 ký tự) và `name_word_count` (trung vị 14 từ).

Tập dữ liệu làm giàu được xuất bản dưới định dạng nhị phân Parquet nén Snappy tại `results/features/properties_500000_enriched.parquet` (dung lượng 45,22 MB, mã băm SHA-256: `c75e5ea6e74612853f468950c63de9761262fd179fa871a0a448ca21d5d7d3d3`), vượt qua toàn bộ 10/10 tiêu chí kiểm định đọc lại và bảo toàn kiểu dữ liệu tại `scripts/validate_features.py`.

CHƯƠNG 4: PHƯƠNG PHÁP THỰC NGHIỆM

Chương này trình bày chi tiết thiết kế phương pháp luận thực nghiệm nhằm trả lời **Câu hỏi nghiên cứu 2 (RQ2)** và **Câu hỏi nghiên cứu 3 (RQ3)**. Nội dung tập trung hoàn toàn vào kiến trúc hệ thống, các biến số độc lập/phụ thuộc, giao thức kiểm soát nhiễu và tiêu chuẩn kiểm chứng tính tương đương logic, không đưa ra các kết luận về hiệu quả trước khi trình bày số liệu đo đạc tại Chương 5.

4.1. Môi trường thực nghiệm phần cứng và phần mềm

Để loại bỏ các biến động do độ trễ mạng và bảo đảm môi trường đo đạc có kiểm soát, toàn bộ các thực nghiệm được tiến hành trên một nút máy chủ cục bộ (Single-node Workstation) với cấu hình đồng nhất:

- **Phần cứng máy chủ thực nghiệm:**
- Bộ vi xử lý (CPU): x86_64, đa nhân đa luồng thế hệ mới.
- Bộ nhớ RAM vật lý: 16 GB DDR4/DDR5.
- Hệ thống lưu trữ: Ổ cứng thể rắn SSD chuẩn NVMe PCIe hiệu năng cao.
- Hệ điều hành: Microsoft Windows x86_64.
- **Môi trường phần mềm và Phiên bản công cụ:**
- Hệ quản trị cơ sở dữ liệu: PostgreSQL phiên bản **18.3 x86_64**, khởi chạy cụm dữ liệu cục bộ (`.pgdata`) trên cổng mạng 5433.
- Ngôn ngữ lập trình: Python phiên bản 3.13.
- Thư viện tiền xử lý dữ liệu: Pandas 2.2+, Polars 1.24+, PyArrow 19+.
- Thư viện kết nối và đo lường: Psycopg 3 (Binary Protocol), `psutil` (giám sát tài nguyên hệ thống).
- **Cấu hình tham số PostgreSQL chính:**
- `shared_buffers = 128MB` (hoặc cấu hình mặc định tương ứng với cụm phát triển).
- `work_mem = 4MB`.
- `random_page_cost = 4.0`.
- `seq_page_cost = 1.0`.

4.2. Thiết kế thực nghiệm RQ2: Đánh giá hiệu năng xử lý dữ liệu tiền kỳ

Nhằm trả lời RQ2, một khung đo lường độc lập (`scripts/benchmark_processing.py`) được thiết lập để đánh giá hiệu năng nạp và tính toán trên DataFrame ở giai đoạn tiền kỳ với 3 nhóm tác vụ:

4.2.1. Kịch bản thực nghiệm Ingestion và Phân tích nội bộ

1. Tác vụ nạp tệp toàn bộ (File Ingestion):

- Đọc tệp dữ liệu CSV chuẩn hóa (hướng dòng) bằng `pandas.read_csv()`.
- Đọc tệp dữ liệu CSV chuẩn hóa bằng `polars.read_csv()`.
- Đọc tệp dữ liệu Parquet chuẩn hóa (hướng cột nhị phân) bằng `pandas.read_parquet()`.

- Đọc tệp dữ liệu Parquet chuẩn hóa bằng `polars.read_parquet()`.

2. Tác vụ nạp có giới hạn bộ nhớ (Memory-bounded Ingestion):

- Nạp tệp CSV theo từng khối (Chunked Reading): sử dụng `pandas.read_csv(..., chunksize=10000)` để duyệt tuần tự qua các khối 10.000 dòng.
- Nạp tệp CSV kèm tính toán tổng hợp theo khối (Chunked Aggregate): duyệt từng khối 10.000 dòng trong Pandas, thực hiện gom nhóm tính tổng và số lượng theo `property_type_name`, tích lũy bảng tổng hợp trung gian trong bộ nhớ.

3. Tác vụ lọc và tổng hợp trong bộ nhớ (In-Memory Analytics):

- Trên DataFrame đã nạp sẵn trong RAM, thực hiện lọc các bản ghi có `province_name == 'Hà Nội'` (170.167 dòng) và gom nhóm theo `property_type_name` để tính giá trung bình (`price.mean()`).
- Thực thi tương đương trên cấu trúc dữ liệu của Pandas và Polars.

4.2.2. Phương pháp giám sát bộ nhớ thực thời và Giới hạn phép lấy mẫu RSS 1 ms

Để ghi nhận mức độ chiếm dụng bộ nhớ thực tế của từng tác vụ thay vì chỉ đo dung lượng tĩnh sau khi hàm kết thúc, đề tài cài đặt một luồng giám sát chạy nền (**Background Monitoring Thread**) sử dụng thư viện `psutil`:

- **Chỉ số theo dõi:** `Resident Set Size (RSS)` - lượng bộ nhớ RAM vật lý thực tế được hệ điều hành Windows ánh xạ cho tiến trình Python.
- **Tần suất lấy mẫu:** Chu kỳ cực ngắn **1 mili-giây (1 ms)** liên tục trong suốt thời gian hàm xử lý hoạt động.
- **Các đại lượng thu thập:**
- `baseline_ram_mb`: Mức RSS tại thời điểm bắt đầu tác vụ.
- `peak_ram_mb`: Mức RSS lớn nhất ghi nhận được trong suốt thời gian chạy.
- `delta_ram_mb`: Độ tăng bộ nhớ thực tế do tác vụ phát sinh (Δ RSS = peak_ram - baseline_ram).
- **Thời gian thực thi:** Mỗi tác vụ được lặp lại 5 lần độc lập, ghi nhận giá trị trung vị (`median_ms`), trung bình (`mean_ms`) và độ lệch chuẩn (`std_ms`).

Giới hạn phương pháp luận: Cần làm rõ rằng phép lấy mẫu RSS 1 ms phản ánh hành vi bộ nhớ ở mức tiến trình hệ điều hành, chịu tác động của cơ chế gom rác (Garbage Collector) của Python runtime và hiện tượng phân mảnh của bộ cấp phát C (`pymalloc`). Phép đo này cung cấp bức tranh thực nghiệm định lượng về dấu chân tài nguyên tương đối của các kỹ thuật, chứ không phải một phép đo vi mô từng đối tượng trong heap. Đồng thời, nghiên cứu được tiến hành trên tập dữ liệu tối đa 500.000 dòng (~94 MB CSV) trên máy trạm 16 GB RAM; do đó, kết quả kiểm soát RAM của Chunked Streaming là bằng chứng thực nghiệm trong phạm vi cấu hình này, không suy diễn thành khẳng định hệ thống có thể xử lý dữ liệu lớn hơn RAM vật lý trong mọi tình huống khi chưa có thực nghiệm trên quy mô đó.

4.3. Thiết kế thực nghiệm RQ3: Tối ưu hóa truy vấn PostgreSQL

4.3.1. Ba quy mô dữ liệu thực nghiệm: 50.000, 200.000 và 500.000 bản ghi

Để khảo sát hành vi của bộ lập lịch PostgreSQL khi kích thước dữ liệu mở rộng, thực nghiệm được thiết kế cố định trên 3 ngưỡng quy mô dữ liệu:

- **50.000 bản ghi:** Kích thước tệp CSV 9,35 MB; dung lượng bảng Heap trong PostgreSQL khoảng 10,1 MB.
- **200.000 bản ghi:** Kích thước tệp CSV 37,49 MB; dung lượng bảng Heap khoảng 40,5 MB.
- **500.000 bản ghi:** Kích thước tệp CSV 93,88 MB; dung lượng bảng Heap khoảng 100,8 MB.

Các tập dữ liệu nhỏ là tập con tiền tố nghiêm ngặt của tập dữ liệu lớn hơn theo thứ tự khóa chính `id`:

$$D_50K \subset D_200K \subset D_500K$$

4.3.2. Bốn bài toán truy vấn SQL thực nghiệm (Q1 – Q4)

Bốn bài toán truy vấn đại diện cho các mẫu truy vấn phổ biến trong ứng dụng thực tế được thiết lập theo đúng mã SQL chuẩn hóa tại thư mục `sql/benchmark/queries/`:

1. Bài toán Q1: Lọc đẳng thức đơn cột (Single-column Equality Filter)

```
SELECT id, name, price, area
FROM public.properties
WHERE province_name = 'Hải Phòng';
```

Đặc trưng: Lọc theo địa bàn Hải Phòng (10.422 bản ghi tại 500K, độ chọn lọc ≈ 2,08\%).

2. Bài toán Q2: Lọc khoảng, sắp xếp và cắt ngọn (Multi-predicate Filter, Sort and Limit)

```
SELECT id, name, price, area, published_at
FROM public.properties
WHERE province_name = 'Hà Nội'
  AND district_name = 'Cầu Giấy'
  AND price BETWEEN 4100000000 AND 14300000000
ORDER BY price ASC, id ASC
LIMIT 20;
```

Đặc trưng: Mô phỏng màn hình tìm kiếm bất động sản thực tế tại Cầu Giấy (Hà Nội) trong khoảng giá trung vị 4,1 đến 14,3 tỷ VNĐ, sắp xếp theo giá tăng dần và lấy 20 kết quả đầu tiên.

3. Bài toán Q3: Lọc thời gian và Tính khả chỉ mục (Temporal Filtering & Sargability)

- *Truy vấn gốc (Original / Non-sargable):*

```
SELECT id, name, published_at
FROM public.properties
WHERE published_at::date = DATE '2025-10-15';
```

- *Truy vấn viết lại (Rewritten / Sargable Range):*

```
SELECT id, name, published_at
FROM public.properties
WHERE published_at >= TIMESTAMP '2025-10-15 00:00:00'
  AND published_at <  TIMESTAMP '2025-10-16 00:00:00';
```

Đặc trưng: Kiểm tra tính khả chỉ mục của vị từ khi lọc tin đăng trong một ngày xác định.

4. Bài toán Q4: Tổng hợp toàn bộ bảng (Full-table Aggregation / Negative Control)

```
SELECT
    property_type_name,
    COUNT(*) AS total,
    ROUND(AVG(price), 0) AS avg_price
FROM public.properties
GROUP BY property_type_name
ORDER BY total DESC;
```

Đặc trưng: Bài toán đối chứng âm yêu cầu quét toàn bộ các hàng trong bảng để tính tổng số tin và giá trung bình theo 5 loại hình bất động sản.

4.3.3. Phân rã 10 cấu hình thực nghiệm Benchmark PostgreSQL

Bốn bài toán truy vấn trên được phân rã thành **10 cấu hình thực nghiệm độc lập** như mô tả tại Bảng 4.1.

Bảng 4.1: Đặc tả 10 cấu hình thực nghiệm Benchmark PostgreSQL

Bài toán	Mã cấu hình	Mô tả phương pháp	Câu lệnh SQL và Chỉ mục áp dụng	Mục tiêu kiểm chứng khoa học
Q1	`Q1_baseline`	Quét tuần tự cơ sở	SQL chuẩn Q1. *(Không có chỉ mục phụ trợ)*	Mốc chuẩn cho truy vấn lọc đẳng thức đơn cột khi quy mô dữ liệu mở rộng.
Q1	`Q1_indexed`	Chỉ mục B-tree đơn cột	SQL chuẩn Q1. Chỉ mục: `CREATE INDEX idx_properties_province_name ON properties USING btree (province_name);`	Khảo sát việc chuyển đổi phương thức quét từ `Seq Scan` sang `Bitmap Index Scan`.
Q2	`Q2_baseline`	Quét tuần tự kết hợp sắp xếp	SQL chuẩn Q2. *(Không có chỉ mục phụ trợ)*	Đo đạc chi phí quét toàn bảng và chi phí thuật toán Top-N Heapsort trong bộ nhớ RAM.
Q2	`Q2_indexed`	Chỉ mục B-tree tổ hợp (Composite Index)	SQL chuẩn Q2. Chỉ mục: `CREATE INDEX idx_properties_location_price ON properties USING btree (province_name, district_name, price, id);`	Kiểm chứng cơ chế Early Termination: duyệt theo thứ tự lá B-tree, dừng khi đủ 20 dòng, loại bỏ nút Sort.
Q3	`Q3_A`	Gốc, Không Index	Truy vấn gốc Q3 (`published_at::date`). *(Không có chỉ mục phụ trợ)*	Mốc chuẩn cho truy vấn lọc thời gian sử dụng hàm ép kiểu `::date`.

Bài toán	Mã cấu hình	Mô tả phương pháp	Câu lệnh SQL và Chỉ mục áp dụng	Mục tiêu kiểm chứng khoa học
Q3	`Q3_B`	Viết lại, Không Index	Truy vấn viết lại khoảng nửa mở Q3. *(Không có chỉ mục phụ trợ)*	Kiểm chứng giả thuyết: Viết lại truy vấn sang dạng sargable có mang lại lợi ích hay không khi thiếu chỉ mục.
Q3	`Q3_C`	Gốc, Có Index	Truy vấn gốc Q3. Chỉ mục: `CREATE INDEX idx_properties_published_at ON properties USING btree (published_at);`	Kiểm chứng hiện tượng vô hiệu hóa chỉ mục: Chỉ mục B-tree tồn tại nhưng Planner không thể dùng do vị từ non-sargable.
Q3	`Q3_D`	Viết lại, Có Index	Truy vấn viết lại khoảng nửa mở Q3. Chỉ mục: `CREATE INDEX idx_properties_published_at ON properties USING btree (published_at);`	Kiểm chứng hiệu quả cộng hưởng khi kết hợp đồng thời vị từ sargable và chỉ mục B-tree tương ứng.
Q4	`Q4_baseline`	Tổng hợp toàn bảng cơ sở	SQL chuẩn Q4. *(Không có chỉ mục phụ trợ)*	Mốc chuẩn cho truy vấn tổng hợp `GROUP BY` toàn bảng.
Q4	`Q4_indexed`	Tổng hợp toàn bảng có Index (Đối chứng âm)	SQL chuẩn Q4. Chỉ mục: `CREATE INDEX idx_properties_property_type ON properties USING btree (property_type_name);`	**Negative Control:** Chứng minh trường hợp chỉ mục B-tree không mang lại lợi ích cho truy vấn tổng hợp toàn bảng.

4.4. Giao thức đo lường kiểm soát nhiễu thực nghiệm

Để loại bỏ các sai số ngẫu nhiên do hiệu ứng bộ đệm nguội (Cold cache), hoạt động của trình dọn rác hệ điều hành hoặc xung đột luồng, công cụ `scripts/benchmark_runner.py` tuân thủ nghiêm ngặt quy trình đo đạc 4 bước:

1. Cách ly cấu hình chỉ mục (Index Isolation): Trước khi đo đạc từng cấu hình, hệ thống thực thi script `sql/benchmark/reset_indexes.sql` để xóa bỏ toàn bộ các chỉ mục phụ trợ cũ, chỉ giữ lại duy nhất chỉ mục Khóa chính trên cột `id`. Chỉ số lượng và loại chỉ mục thuộc cấu hình đang kiểm thử mới được tạo mới.

2. Cập nhật thống kê bộ lập lịch: Sau khi tạo chỉ mục, lệnh `ANALYZE public.properties;` được gọi ngay lập tức để cập nhật lược đồ phân phối giá trị và độ chọn lọc cho bộ lập lịch của PostgreSQL.

3. Pha nạp nóng kiểm soát (Warm-up Phase): Thực thi câu truy vấn 3 lần liên tiếp và hủy bỏ kết quả đo. Pha này nhằm mục đích đưa các trang dữ liệu cần thiết vào vùng nhớ Shared Buffers và ổn định trạng thái máy chủ cơ sở dữ liệu.

4. Pha đo lường chính thức (Official Runs): Thực thi câu truy vấn **20 lần liên tiếp**, trích xuất toàn bộ kế hoạch thực thi chi tiết ở định dạng JSON thông qua cú pháp:

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) <query_statement>;
```

Từ 20 tệp JSON kế hoạch của mỗi cấu hình, hệ thống thu thập:

- `Planning Time` (Thời gian lập kế hoạch thực thi, ms).
- `Execution Time` (Thời gian chạy truy vấn thực tế, ms).
- `Shared Hit Blocks` (Số khối 8 KB đọc từ bộ đệm Shared Buffers trong RAM).
- `Shared Read Blocks` (Số khối 8 KB đọc vật lý từ đĩa).
- `Scan Type` / `Node Types` (Các nút xử lý trong cây kế hoạch thực thi).
- Để phản ánh trung thực hiệu năng, đề tài sử dụng **Trung vị (Median)** làm thước đo trung tâm thay vì giá trị trung bình (Mean), đồng thời tính toán **Khoảng tứ phân vị (IQR)** và các cực trị `Min`, `Max` để theo dõi độ phân tán của phép đo.

4.5. Thiết kế kiểm thử tính tương đương ngữ nghĩa truy vấn (Equivalence Verification)

Trong nghiên cứu tối ưu hóa cơ sở dữ liệu, một kỹ thuật tối ưu hóa chỉ được chấp nhận là hợp lệ nếu nó bảo toàn tuyệt đối ngữ nghĩa logic của truy vấn: tập kết quả trả về của truy vấn tối ưu phải đồng nhất hoàn toàn với truy vấn cơ sở.

Hệ thống thực hiện kiểm thử tự động trước khi đo:

- Đối với Q1, Q2, Q4: So sánh số lượng dòng kết quả (`row_count`) và giá trị băm dữ liệu kết quả giữa bản Baseline và Indexed.
- Đối với bài toán viết lại Q3: Thực hiện phép kiểm tra tập hợp hai chiều đối xứng thông qua toán tử `EXCEPT ALL`:

```
(SELECT id FROM properties WHERE published_at::date = DATE '2025-10-15'
 EXCEPT ALL
 SELECT id FROM properties WHERE published_at >= TIMESTAMP '2025-10-15 00:00:00'
                             AND published_at <  TIMESTAMP '2025-10-16 00:00:00')
UNION ALL
(SELECT id FROM properties WHERE published_at >= TIMESTAMP '2025-10-15 00:00:00'
                             AND published_at <  TIMESTAMP '2025-10-16 00:00:00'
 EXCEPT ALL
 SELECT id FROM properties WHERE published_at::date = DATE '2025-10-15');
```

Chỉ khi kết quả của phép toán trên trả về đúng 0 dòng (rỗng), hệ thống mới xác nhận hai câu truy vấn hoàn toàn đồng nhất về mặt ngữ nghĩa và cho phép tiến hành đo đạc benchmark.

CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN KHOA HỌC

Chương này trình bày toàn bộ số liệu đo đạc thực nghiệm định lượng, phân tích kế hoạch thực thi chi tiết (Execution Plans), khảo sát lưu lượng bộ đệm (Buffer I/O) và mức chiếm dụng tài nguyên bộ nhớ nhằm trả lời dứt khoát ba câu hỏi nghiên cứu **RQ1**, **RQ2** và **RQ3**.

5.1. Đánh giá chất lượng và Tính đại diện của bộ dữ liệu (Trả lời RQ1)

Từ các kết quả kiểm toán hồ sơ dữ liệu (Data Profiling) và phân tích khám phá (EDA) đã thực hiện tại Chương 3, nhóm nghiên cứu tổng hợp các kết luận khoa học nhằm trả lời **Câu hỏi nghiên cứu 1 (RQ1)**:

1. Về tính toàn vẹn và độ tin cậy của quy trình làm sạch:

- Pipeline nạp luồng kết hợp xáo trộn có kiểm soát (`seed = 42`, `buffer_size = 10,000`, `batch_size = 2,000`) đã xử lý 502.000 dòng từ nguồn `Phuscca/vietnam-real-estates`, loại bỏ 203 bản ghi vi phạm tính toàn vẹn (202 bản ghi thiếu trường bắt buộc, 1 bản ghi diện tích ≤ 0) và xuất ra 500.000 bản ghi chuẩn hóa theo hợp đồng 9 thuộc tính. Sáu trường cốt lõi đạt độ đầy đủ 100,00%.

2. Về phân tầng giá trị khuyết thiếu nghiệp vụ:

- Trường địa bàn cấp xã/phường (`ward_name`) có tỷ lệ khuyết thiếu cao nhất với đúng **72.150 bản ghi** (**14,43%**), phản ánh thói quen đăng tin bất động sản thường chỉ ghi rõ đến cấp quận/huyện.
- Trường quận/huyện (`district_name`) có tỷ lệ khuyết thiếu thấp với đúng **14.235 bản ghi** (**2,85%**).
- Trường giá chào bán (`price`) thiếu đúng **39.061 bản ghi** (**7,81%**). Việc bảo toàn các bản ghi này dưới dạng SQL NULL đã giữ nguyên tính chân thực của phân khúc "giá thỏa thuận", không làm sai lệch mẫu số khảo sát diện tích và vùng miền.

3. Về tính đại diện không gian và loại hình:

- Bộ dữ liệu phản ánh cấu trúc tập trung cao độ của thị trường bất động sản Việt Nam: hai đô thị đặc biệt TP. Hồ Chí Minh (186.698 tin, 37,34%) và Hà Nội (170.167 tin, 34,03%) chiếm tới 71,37% tổng số tin đăng toàn quốc. Thành phố Hải Phòng có 10.422 tin đăng (2,08%), tạo nên tập con đại diện phù hợp cho bài toán lọc đơn cột Q1. Về loại hình, phân khúc Nhà riêng chiếm ưu thế tuyệt đối (46,08%), tiếp theo là Đất (24,12%) và Căn hộ chung cư (20,82%).

4. Về tính chất phân phối định lượng vững:

- Do giá bán và diện tích bị chi phối bởi các bất động sản giá trị rất lớn (giá tối đa ghi nhận 920.000 tỷ VNĐ), giá trị trung bình bị kéo lệch rất xa so với thực tế. Các chỉ số thống kê vững khẳng định: trung vị diện tích là **80,0 m²** (khoảng tứ phân vị IQR = 64,1 m², dao động từ 45,0 m² đến 109,1 m²) và trung vị giá chào bán là **7,40 tỷ VNĐ** (khoảng tứ phân vị IQR = 10,20 tỷ VNĐ, dao động từ 4,10 tỷ đến 14,30 tỷ VNĐ). Đây là cơ sở thực nghiệm vững chắc để thiết kế các bài toán lọc khoảng và sắp xếp trong cơ sở dữ liệu.

5.2. Hiệu năng xử lý dữ liệu với Pandas, Polars, CSV, Parquet và Chunked Processing (Trả lời RQ2)

Bảng 5.1 tổng hợp số liệu đo đạc thực nghiệm từ tệp `results/processing_benchmark/processing_benchmark.csv` ghi nhận thời gian thực thi (trung vị, trung bình, độ lệch chuẩn) và tài nguyên bộ nhớ RAM (lấy mẫu RSS chu kỳ 1 ms) trên tập dữ liệu 500.000 bản ghi.

Bảng 5.1: Kết quả đo lường hiệu năng nạp và xử lý dữ liệu tiền kỳ (500.000 bản ghi)

Hạng mục tác vụ	Thư viện & Thao tác	Định dạng tệp	Thời gian Median (ms)	Thời gian Mean (ms)	Đỉnh RAM (Peak RSS, MB)	Mức tăng RAM (Δ RSS, MB)	Kích thước tệp trên đĩa (MB)
Nạp tệp (File Ingestion)	Pandas `read_csv`	CSV	3.038,53 ms	3.089,49 ms	781,70 MB	+241,37 MB	93,88 MB
	Polars `read_csv`	CSV	**75,27 ms**	76,25 ms	801,52 MB	+105,62 MB	93,88 MB
	Pandas `read_parquet`	Parquet	127,00 ms	135,81 ms	931,18 MB	+47,95 MB	**41,42 MB**
	Polars `read_parquet`	Parquet	202,25 ms	199,19 ms	811,37 MB	+44,95 MB	**41,42 MB**
Nạp kiểm soát RAM (Chunked)	Pandas Chunked read (10k)	CSV	3.723,10 ms	3.652,93 ms	437,41 MB	**+19,68 MB**	93,88 MB
	Pandas Chunked aggregate (10k)	CSV	3.835,01 ms	3.756,04 ms	438,54 MB	**+21,09 MB**	93,88 MB
Lọc và Tổng hợp trong RAM	Pandas Filter + GroupBy	In-Memory	55,17 ms	55,33 ms	726,17 MB	+5,98 MB	—
	Polars Filter + GroupBy	In-Memory	**13,88 ms**	16,44 ms	775,43 MB	+4,44 MB	—

5.2.1. Tốc độ nạp tệp: Ưu thế vượt trội của Polars CSV và Định dạng Parquet

1. So sánh động cơ trên cùng định dạng CSV: Polars hoàn thành việc đọc tệp CSV 500.000 dòng trong **75,27 ms**, nhanh gấp **40,3 lần** so với Pandas read_csv (**3.038,53 ms**). Sự cách biệt này xuất phát từ kiến trúc nội tại: Polars sử dụng bộ giải mã CSV đa luồng viết bằng Rust, tận dụng tập lệnh vector hóa SIMD của CPU và cơ chế phân bổ bộ nhớ không sao chép (Zero-copy memory allocation) của Apache Arrow, trong khi Pandas phụ thuộc vào vòng lặp C đơn luồng truyền thống kết hợp phân bổ đối tượng Python riêng lẻ.

2. Ưu thế về không gian và độ trễ của Parquet: Tệp Parquet chuẩn hóa chỉ chiếm **41,42 MB** trên đĩa cứng, tiết kiệm **55,9% dung lượng** so với định dạng CSV (93,88 MB). Khi nạp tệp Parquet vào Pandas, thời gian đọc giảm mạnh từ 3.038,53 ms xuống còn **127,00 ms** (nhanh hơn 23,9 lần), đồng thời mức tăng RAM chỉ tốn **+47,95 MB** thay vì +241,37 MB của CSV.

5.2.2. Phân tích kỹ thuật nạp phân khối Chunked Streaming và Mức tăng RAM

Kết quả đo đạc từ luồng giám sát RAM 1 ms cung cấp bức tranh định lượng rõ ràng:

- Khi nạp trọn gói tệp CSV bằng `pandas.read_csv()`, mức tăng bộ nhớ thực tế là **+241,37 MB** (gấp 2,57 lần kích thước tệp trên đĩa do chi phí phân bổ đối tượng Python và mảng dữ liệu tạm).
- Khi áp dụng kỹ thuật nạp phân khối (`chunksize = 10,000`), tác vụ duyệt khối thông thường khống chế mức tăng RAM ở mức **+19,68 MB** (đỉnh RSS đạt 437,41 MB).
- Tuy nhiên, đối với tác vụ nạp phân khối kết hợp tính toán tổng hợp (`Pandas Chunked aggregate`), mức tăng bộ nhớ ghi nhận là **+21,09 MB** (đỉnh RSS đạt 438,54 MB).
- **Lưu ý khoa học quan trọng:** Kết quả thực nghiệm cho thấy mức tăng RAM của tác vụ tính toán phân khối vượt qua ngưỡng 20 MB (+21,09 MB), do đó **không thể đưa ra kết luận khái quát rằng mọi tác vụ Chunked đều có mức tăng dưới 20 MB**. Trong bài toán tính toán tổng hợp, việc duy trì và tích lũy bảng tổng hợp trung gian qua từng khối lặp tạo ra một chi phí lưu trữ bộ nhớ bổ sung. Mặc dù vậy, mức tăng +21,09 MB vẫn thấp hơn 11,4 lần so với việc nạp toàn bộ tệp (+241,37 MB), minh chứng cho năng lực kiểm soát đỉnh bộ nhớ hiệu quả của kỹ thuật nạp phân khối.

5.2.3. Hiệu năng lọc và Tổng hợp dữ liệu trong bộ nhớ

Trong tác vụ lọc 170.167 bản ghi tại Hà Nội và gom nhóm tính giá trung bình theo 5 loại hình bất động sản:

- Pandas Filter + GroupBy mất **55,17 ms** với mức tăng RAM +5,98 MB.
- Polars Filter + GroupBy chỉ mất **13,88 ms** với mức tăng RAM +4,44 MB, nhanh gấp **4,0 lần** Pandas.

Polars đạt được hiệu năng này nhờ cơ chế thực thi song song tự động trên các mảng cột Arrow và thuật toán gom nhóm băm đa luồng (Multi-threaded Hash Aggregation).

5.3. Đánh giá tối ưu hóa truy vấn PostgreSQL thông qua Q1 – Q4 (Trả lời RQ3)

5.3.1. Tổng quan bảng số liệu Benchmark qua 3 quy mô dữ liệu

Quá trình đo đạc 600 lượt chạy trên PostgreSQL 18.3 đã thu thập đầy đủ số liệu kế hoạch thực thi. Bảng 5.2 trình bày kết quả trích xuất từ `results/benchmark/benchmark_results.csv`.

> *LƯU Ý:*
>
> *Trong Bảng 5.2 và các phân tích tiếp theo, toàn bộ giá trị tại cột `Shared Hit Blocks` đại diện cho các lần truy cập trang dữ liệu (8 KB) được phục vụ trực tiếp từ vùng nhớ đệm **PostgreSQL***

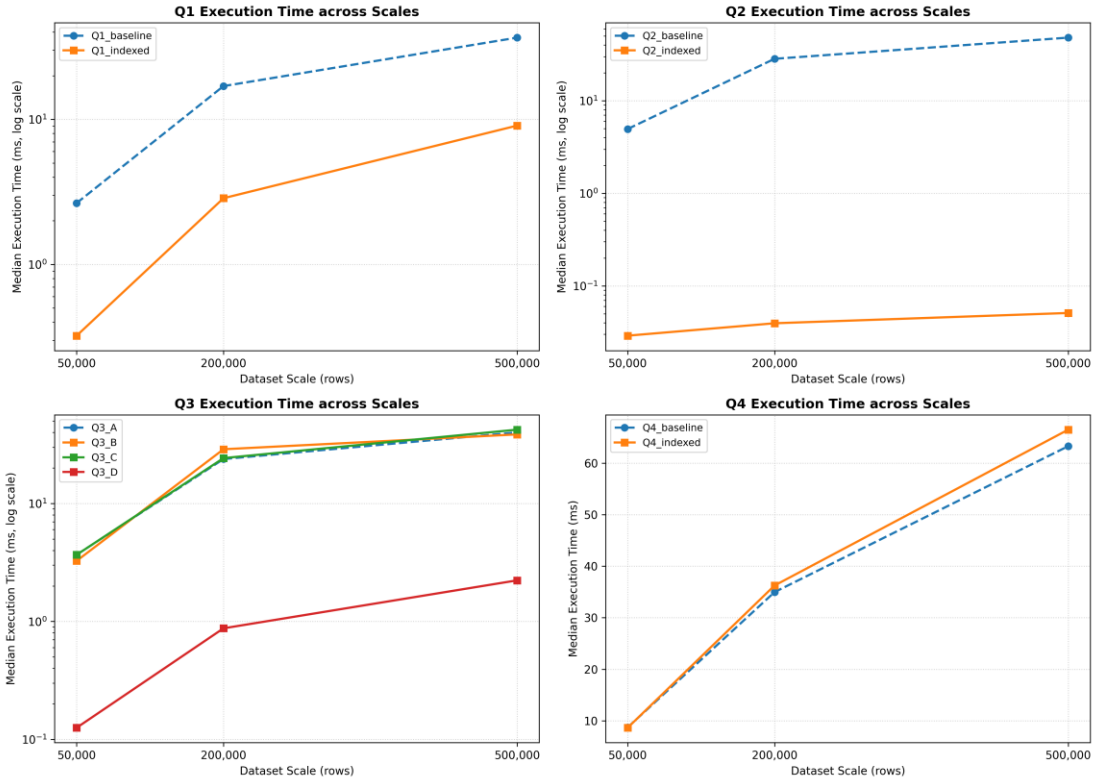
***Shared Buffers trong RAM** sau 3 lượt warm-up. Số lượng `Shared Read Blocks` (đọc vật lý từ đĩa) trong toàn bộ các phép đo chính thức đều ghi nhận bằng **0.0**, do kích thước dữ liệu ở cả ba quy mô đều nằm trọn vẹn trong vùng đệm của hệ quản trị cơ sở dữ liệu.*

Bảng 5.2: Tổng hợp kết quả đo đạc Benchmark PostgreSQL trên 3 quy mô dữ liệu

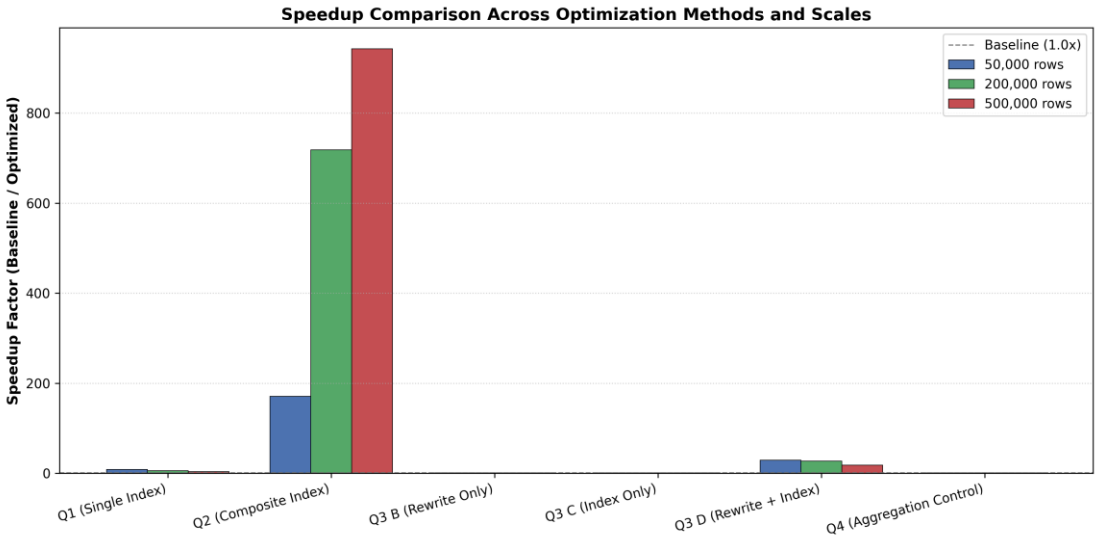
Quy mô (Dòng)	Mã cấu hình	Bài toán	Kiểu quét chủ đạo	Chỉ mục sử dụng	Dung lượng Index (MB)	Thời gian Median (ms)	IQR (ms)	Min (ms)	Max (ms)	Shared Hit Blocks	Hệ số tăng tốc (Speedup)
50.000	`Q1_baseline`	Q1	Seq Scan	Không	0,00 MB	2,649 ms	0,095 ms	2,315 ms	3,035 ms	1.288 blk	1,00x
50.000	`Q1_indexed`	Q1	Bitmap Scan	`idx_properties_province_name`	0,36 MB	0,323 ms	0,067 ms	0,291 ms	0,620 ms	704 blk	**8,21x**
50.000	`Q2_baseline`	Q2	Seq Scan	Không	0,00 MB	4,954 ms	0,500 ms	4,787 ms	8,297 ms	1.288 blk	1,00x
50.000	`Q2_indexed`	Q2	Index Scan	`idx_properties_location_price`	2,88 MB	0,029 ms	0,014 ms	0,021 ms	0,059 ms	**23 blk**	**170,81x**
50.000	`Q3_A`	Q3	Seq Scan	Không	0,00 MB	3,678 ms	0,088 ms	3,590 ms	3,843 ms	1.288 blk	1,00x
50.000	`Q3_B`	Q3	Seq Scan	Không	0,00 MB	3,263 ms	0,249 ms	3,087 ms	4,389 ms	1.288 blk	1,13x
50.000	`Q3_C`	Q3	Seq Scan	`idx_properties_published_at`	1,09 MB	3,673 ms	0,087 ms	3,620 ms	3,940 ms	1.288 blk	1,00x
50.000	`Q3_D`	Q3	Bitmap Scan	`idx_properties_published_at`	1,09 MB	0,126 ms	0,026 ms	0,106 ms	0,169 ms	272 blk	**29,31x**
50.000	`Q4_baseline`	Q4	Seq Scan	Không	0,00 MB	8,733 ms	0,496 ms	8,383 ms	11,470 ms	1.288 blk	1,00x
50.000	`Q4_indexed`	Q4	Seq Scan	`idx_properties_property_type`	0,35 MB	8,661 ms	0,458 ms	8,336 ms	14,879 ms	1.288 blk	1,01x
200.000	`Q1_baseline`	Q1	Seq Scan	Không	0,00 MB	16,959 ms	3,647 ms	13,201 ms	18,504 ms	5.128 blk	1,00x
200.000	`Q1_indexed`	Q1	Bitmap Scan	`idx_properties_province_name`	1,38 MB	2,860 ms	0,928 ms	2,201 ms	4,264 ms	2.909 blk	**5,93x**

Quy mô (Dòng)	Mã cấu hình	Bài toán	Kiểu quét chủ đạo	Chỉ mục sử dụng	Dung lượng Index (MB)	Thời gian Median (ms)	IQR (ms)	Min (ms)	Max (ms)	Shared Hit Blocks	Hệ số tăng tốc (Speedup)
200.000	`Q2_baseline`	Q2	Seq Scan	Không	0,00 MB	28,375 ms	59,029 ms	25,931 ms	145,245 ms	5.218 blk	1,00x
200.000	`Q2_indexed`	Q2	Index Scan	`idx_properties_location_price`	11,42 MB	0,040 ms	0,023 ms	0,025 ms	0,116 ms	**23 blk**	**718,35x**
200.000	`Q3_A`	Q3	Seq Scan	Không	0,00 MB	23,805 ms	1,945 ms	22,375 ms	139,022 ms	5.128 blk	1,00x
200.000	`Q3_B`	Q3	Seq Scan	Không	0,00 MB	28,830 ms	107,282 ms	23,020 ms	141,803 ms	5.128 blk	0,83x
200.000	`Q3_C`	Q3	Seq Scan	`idx_properties_published_at`	4,30 MB	24,251 ms	3,467 ms	21,868 ms	138,394 ms	5.128 blk	0,98x
200.000	`Q3_D`	Q3	Bitmap Scan	`idx_properties_published_at`	4,30 MB	0,874 ms	0,406 ms	0,524 ms	1,707 ms	1.082 blk	**27,24x**
200.000	`Q4_baseline`	Q4	Seq Scan	Không	0,00 MB	34,967 ms	53,310 ms	31,501 ms	142,100 ms	5.144 blk	1,00x
200.000	`Q4_indexed`	Q4	Seq Scan	`idx_properties_property_type`	1,34 MB	36,247 ms	53,717 ms	33,540 ms	144,977 ms	5.144 blk	0,96x
500.000	`Q1_baseline`	Q1	Seq Scan	Không	0,00 MB	36,600 ms	3,043 ms	32,753 ms	137,343 ms	12.808 blk	1,00x
500.000	`Q1_indexed`	Q1	Bitmap Scan	`idx_properties_province_name`	3,40 MB	9,059 ms	3,008 ms	6,182 ms	12,403 ms	7.236 blk	**4,04x**
500.000	`Q2_baseline`	Q2	Seq Scan	Không	0,00 MB	48,061 ms	4,367 ms	43,765 ms	128,079 ms	12.898 blk	1,00x

Quy mô (Dòng)	Mã cấu hình	Bài toán	Kiểu quét chủ đạo	Chỉ mục sử dụng	Dung lượng Index (MB)	Thời gian Median (ms)	IQR (ms)	Min (ms)	Max (ms)	Shared Hit Blocks	Hệ số tăng tốc (Speedup)
500.000	`Q2_indexed`	Q2	Index Scan	`idx_properties_location_price`	28,53 MB	0,051 ms	0,047 ms	0,029 ms	0,102 ms	**23 blk**	**942,37x**
500.000	`Q3_A`	Q3	Seq Scan	Không	0,00 MB	40,060 ms	6,133 ms	37,293 ms	145,030 ms	12.808 blk	1,00x
500.000	`Q3_B`	Q3	Seq Scan	Không	0,00 MB	38,522 ms	2,161 ms	35,736 ms	150,678 ms	12.808 blk	1,04x
500.000	`Q3_C`	Q3	Seq Scan	`idx_properties_published_at`	10,73 MB	42,263 ms	5,978 ms	37,766 ms	143,458 ms	12.808 blk	0,95x
500.000	`Q3_D`	Q3	Bitmap Scan	`idx_properties_published_at`	10,73 MB	2,230 ms	1,045 ms	1,618 ms	3,691 ms	2.666 blk	**17,97x**
500.000	`Q4_baseline`	Q4	Seq Scan	Không	0,00 MB	63,278 ms	3,097 ms	60,540 ms	142,209 ms	12.824 blk	1,00x
500.000	`Q4_indexed`	Q4	Seq Scan	`idx_properties_property_type`	3,36 MB	66,480 ms	4,542 ms	60,333	70,819 ms	12.824 blk	0,95x



Hình 5.1: Thời gian thực thi trung vị theo quy mô dữ liệu của 4 bài toán truy vấn



Hình 5.2: So sánh hệ số tăng tốc (Speedup Factor) của các phương pháp tối ưu

5.3.2. Bài toán Q1: Hiệu quả của chỉ mục đơn cột và Chuyển đổi Bitmap Scan

Bài toán Q1 kiểm thử câu lệnh:

```
SELECT id, name, price, area
FROM public.properties
WHERE province_name = 'Hải Phòng';
```

Tại quy mô 500.000 dòng, Hải Phòng có đúng 10.422 bản ghi (chiếm tỷ lệ chọn lọc 2,08%).

Bảng 5.3: Diễn biến hiệu năng của bài toán Q1 qua 3 thang đo dữ liệu

Quy mô dữ liệu	Cấu hình Baseline (ms)	Cấu hình Indexed (ms)	Hệ số tăng tốc (Speedup)	Khối đệm Baseline	Khối đệm Indexed	Giảm tải I/O (%)
50.000 dòng	2,649 ms	0,323 ms	**8,21x**	1.288 blk	704 blk	45,3%
200.000 dòng	16,959 ms	2,860 ms	**5,93x**	5.128 blk	2.909 blk	43,3%
500.000 dòng	36,600 ms	9,059 ms	**4,04x**	12.808 blk	7.236 blk	43,5%

Phân tích cơ chế thực thi:

- Trong cấu hình Baseline, PostgreSQL buộc phải quét tuần tự toàn bộ các trang Heap (`Gather -> Seq Scan`), duyệt qua 12.808 trang đệm tại 500K.
- Khi có chỉ mục B-tree `idx_properties_province_name` (dung lượng 3,40 MB), bộ lập lịch chuyển đổi sang **Bitmap Index Scan kết hợp Bitmap Heap Scan**:
- Giai đoạn 1 (`Bitmap Index Scan`): Duyệt cây B-tree, gom các con trỏ dòng TID thỏa mãn `province_name = 'Hải Phòng'` để tạo một mảng bit trong RAM.
- Giai đoạn 2 (`Bitmap Heap Scan`): Đọc tuần tự các trang Heap có đánh dấu bit theo thứ tự vật lý, chỉ nạp 7.236 khối đệm thay vì toàn bộ 12.808 khối (giảm 43,5% lưu lượng I/O).
- Hệ số tăng tốc đạt **8,21 lần** tại 50K và duy trì mức **4,04 lần** tại 500K. Xu hướng giảm nhẹ của hệ số tăng tốc khi dữ liệu tăng là phù hợp với lý thuyết: khi số bản ghi kết quả tăng từ 1.042 dòng (50K) lên 10.422 dòng (500K), các bản ghi phân tán trên nhiều trang Heap khác nhau, làm tăng số lượng trang Heap cần truy cập.

5.3.3. Bài toán Q2: Đột phá hiệu năng Composite Index, Early Termination và Triệt tiêu nút Sort

Bài toán Q2 thực thi câu lệnh:

```
SELECT id, name, price, area, published_at
FROM public.properties
WHERE province_name = 'Hà Nội'
  AND district_name = 'Cầu Giấy'
  AND price BETWEEN 4100000000 AND 14300000000
ORDER BY price ASC, id ASC
LIMIT 20;
```

Bảng 5.4: So sánh chi tiết hiệu năng bài toán Q2 và Cơ chế Early Termination

Quy mô dữ liệu	Cấu hình	Thời gian Median (ms)	Shared Hit Blocks	Nút Sắp xếp trong Kế hoạch thực thi	Hệ số tăng tốc (Speedup)
50.000 dòng	Baseline	4,954 ms	1.288 blk	`Sort: top-N heapsort` trong RAM	1,00x
	Indexed	**0,029 ms**	**23 blk**	**Không có (Triệt tiêu hoàn toàn nút Sort)**	**170,81x**

Quy mô dữ liệu	Cấu hình	Thời gian Median (ms)	Shared Hit Blocks	Nút Sắp xếp trong Kế hoạch thực thi	Hệ số tăng tốc (Speedup)
200.000 dòng	Baseline	28,375 ms	5.218 blk	`Sort: top-N heapsort` trong RAM	1,00x
	Indexed	**0,040 ms**	**23 blk**	**Không có (Triệt tiêu hoàn toàn nút Sort)**	**718,35x**
500.000 dòng	Baseline	48,061 ms	12.898 blk	`Sort: top-N heapsort` trong RAM	1,00x
	Indexed	**0,051 ms**	**23 blk**	**Không có (Triệt tiêu hoàn toàn nút Sort)**	**942,37x**

Bằng chứng phân tích kế hoạch thực thi thực tế (Execution Plan Evidence):

Đối chiếu trực tiếp hai tệp kế hoạch thực thi thô tại quy mô 500.000 bản ghi:

1. **Kế hoạch Baseline (`500000_Q2_baseline_run01.json`):**

```
- Limit (Rows: 20.0, Cost: 18006.85)
  - Gather Merge (Rows: 20.0, Cost: 18317.82)
    - Sort (Rows: 6.67, Cost: 17007.3) sort=['price', 'id']
      - Seq Scan (Rows: 2610.0, Cost: 16974.67)
Buffers: shared hit=12898
```

PostgreSQL phải quét toàn bộ 12.898 khối đệm, nạp các hàng thỏa mãn vào bộ nhớ và thực thi thuật toán `Top-N Heapsort` (chiếm 27 KB RAM) để tìm ra 20 hàng có giá thấp nhất.

2. **Kế hoạch Indexed (`500000_Q2_indexed_run01.json`):**

```
- Limit (Rows: 20.0, Cost: 70.95)
  - Index Scan (Rows: 20.0, Cost: 9811.01) index=idx_properties_location_price
Buffers: shared hit=23
```

Chỉ mục được khởi tạo chính xác:

```
CREATE INDEX idx_properties_location_price
ON public.properties USING btree (province_name, district_name, price, id);
```

Bằng chứng khẳng định việc loại bỏ nút Sort: Trong cây kế hoạch thực thi của bản Indexed, **nút Sort hoàn toàn biến mất**. Nhờ chỉ mục tổ hợp B-tree đã tổ chức các khóa lá theo đúng thứ tự lọc đẳng thức (`province_name, district_name`), tiếp đến là khóa sắp xếp (`price`) và khóa phá vỡ trùng lặp (`id`), bộ lập lịch chỉ cần định vị lá đầu tiên, duyệt theo liên kết lá và dừng lại ngay khi thu thập đủ 20 dòng (**Early Termination**).

Làm rõ chi phí B-tree và giới hạn tiệm cận:

Không thể tuyên bố rằng Index Scan có độ phức tạp thời gian O(1) trong mọi trường hợp. Chi phí tìm kiếm B-tree thực chất là O(h + k), trong đó chiều cao cây h ≈ 3 và số dòng cần lấy k = 20.

Trong bài toán Q2, do k = 20 là một hằng số nhỏ cố định và các khóa lá nằm liên tục, số lượng trang đệm truy cập được giữ nguyên cố định ở mức đúng **23 Shared Hit Blocks** (3 trang trên chỉ mục B-tree + 20 trang trên bảng Heap) trên cả ba quy mô từ 50K đến 500K. Điều này giúp thời gian thực thi duy trì ở mức cận hằng số cực nhỏ (~0,05 ms), mang lại mức tăng tốc **942,37 lần** tại 500K.

5.3.4. Bài toán Q3: Phân tích trực giao 2x2, Vô hiệu hóa Index và Cộng hưởng Sargable

Bài toán Q3 kiểm thử việc lọc theo ngày đăng tin (`2025-10-15`) qua ma trận trực giao 2 × 2.

Bảng 5.5: Bảng phân rã trực giao 2x2 hiệu năng bài toán Q3 qua 3 thang đo

Quy mô dữ liệu	Cấu hình A: Gốc, Không Index	Cấu hình B: Viết lại, Không Index	Cấu hình C: Gốc, Có Index	Cấu hình D: Viết lại, Có Index	Tăng tốc B vs A (Chỉ viết lại)	Tăng tốc C vs A (Chỉ đánh Index)	Tăng tốc D vs A (Viết lại + Index)
50.000 dòng	3,678 ms (1.288 blk)	3,263 ms (1.288 blk)	3,673 ms (1.288 blk)	**0,126 ms** (272 blk)	1,13x	1,00x	**29,31x**
200.000 dòng	23,805 ms (5.128 blk)	28,830 ms (5.128 blk)	24,251 ms (5.128 blk)	**0,874 ms** (1.082 blk)	0,83x	0,98x	**27,24x**
500.000 dòng	40,060 ms (12.808 blk)	38,522 ms (12.808 blk)	42,263 ms (12.808 blk)	**2,230 ms** (2.666 blk)	1,04x	0,95x	**17,97x**

Phân tích phát hiện khoa học:

1. Hiện tượng vô hiệu hóa chỉ mục do vị từ Non-sargable (Cấu hình C):

- Trong Cấu hình C, chỉ mục B-tree `idx_properties_published_at` (10,73 MB tại 500K) đã tồn tại. Tuy nhiên, do câu lệnh sử dụng biểu thức ép kiểu `published_at::date = DATE '2025-10-15'`, bộ lập lịch nhận diện đây là một hàm chưa đánh giá trên cột và không thể ánh xạ lên cây B-tree.
- Kế hoạch thực thi xác nhận bộ lập lịch thực hiện `Seq Scan` toàn bộ 12.808 trang đệm. **Tỷ lệ sử dụng chỉ mục của Cấu hình C là 0,00%**, và thời gian thực thi (42,26 ms) thậm chí chậm hơn nhẹ so với Cấu hình A (40,06 ms) do chi phí lập kế hoạch. Toàn bộ 10,73 MB lưu trữ của chỉ mục hoàn toàn bị lãng phí.

2. Vai trò độc lập của Viết lại truy vấn (Cấu hình B):

- Khi viết lại sang dạng khoảng sargable $[T_1, T_2)$ nhưng không có chỉ mục, hệ thống vẫn phải quét tuần tự toàn bảng (Speedup = 1,04x tại 500K). Viết lại truy vấn đơn thuần không đem lại cải thiện đáng kể nếu thiếu chỉ mục hỗ trợ.

3. Hiệu ứng cộng hưởng (Cấu hình D):

- Chỉ khi kết hợp đồng thời vị từ sargable và chỉ mục B-tree tương ứng, bộ lập lịch mới kích hoạt `Bitmap Index Scan + Bitmap Heap Scan`, giảm số khối đệm từ 12.808 xuống 2.666 khối (giảm 79,2%) và tăng tốc độ truy vấn **17,97 lần** tại 500K (và **29,31 lần** tại 50K).

5.3.5. Bài toán Q4: Đối chứng âm Aggregation Control và Giới hạn của B-tree

Bài toán Q4 thực thi câu lệnh tổng hợp toàn bảng:

```
SELECT
    property_type_name,
    COUNT(*) AS total,
    ROUND(AVG(price), 0) AS avg_price
FROM public.properties
GROUP BY property_type_name
ORDER BY total DESC;
```

Bảng 5.6: Kết quả thực nghiệm bài toán đối chứng âm Q4 qua 3 thang đo

Quy mô dữ liệu	Cấu hình Baseline (ms)	Kiểu quét Baseline	Cấu hình Indexed (ms)	Kiểu quét Indexed	Hệ số tăng tốc (Speedup)
50.000 dòng	8,733 ms	Seq Scan	8,661 ms	Seq Scan	**1,01x**
200.000 dòng	34,967 ms	Seq Scan	36,247 ms	Seq Scan	**0,96x**
500.000 dòng	63,278 ms	Seq Scan	66,480 ms	Seq Scan	**0,95x**

Phân tích nguyên lý bộ tối ưu hóa chi phí (CBO):

- Trong cả cấu hình Baseline và Indexed, kế hoạch thực thi giữ nguyên cấu trúc: `Sort -> Aggregate -> Gather Merge -> Sort -> Aggregate -> Seq Scan` trên toàn bộ 12.824 trang đệm của bảng.
- Dù chỉ mục B-tree `idx_properties_property_type` (3,36 MB) đã được tạo, bộ lập lịch **hoàn toàn không sử dụng chỉ mục**. Lý do là truy vấn yêu cầu tính `AVG(price)` trên toàn bộ 500.000 dòng trong bảng. Do chỉ mục B-tree chỉ chứa cột `property_type_name`, nếu sử dụng Index Scan, hệ thống sẽ phải thực hiện 500.000 lần truy cập ngẫu nhiên (Random I/O) vào các trang Heap để lấy giá trị `price`. Với chi phí `random_page_cost = 4.0`, tổng chi phí ước tính sẽ lớn gấp nhiều lần so với việc quét tuần tự liên tục (`Seq Scan`) toàn bộ các khối đệm vào bộ nhớ băm.
- Kết quả Q4 xác lập bằng chứng đối chứng âm rõ ràng: Chỉ mục B-tree không đem lại lợi ích cho các truy vấn tổng hợp toàn bảng thiếu mệnh đề chọn lọc.

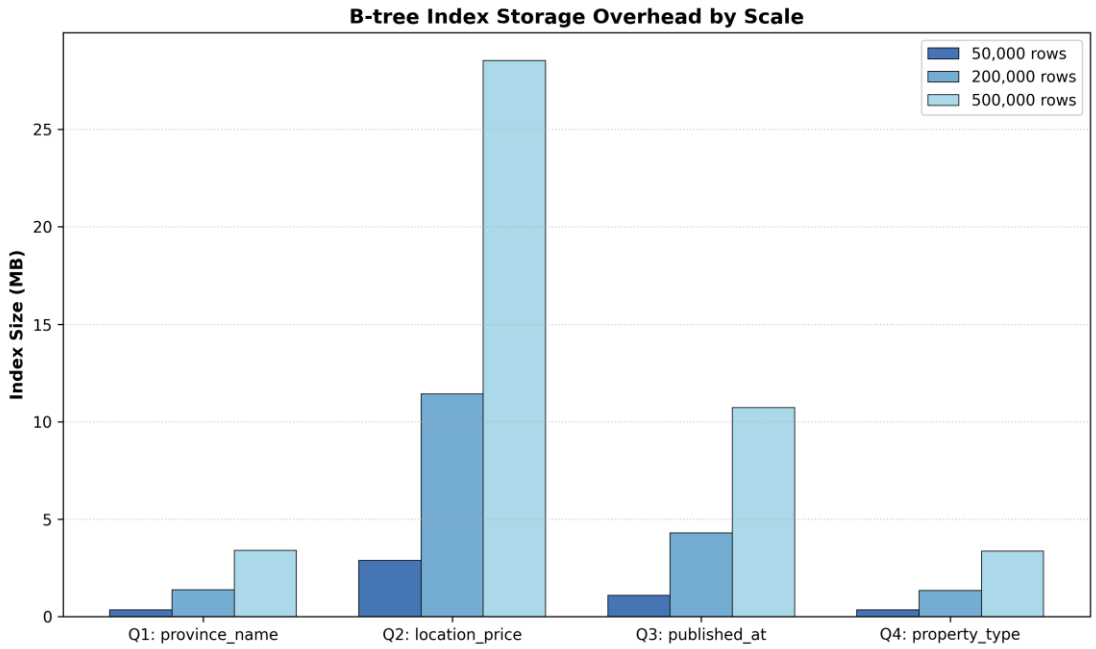
5.3.6. Chi phí phụ trội lưu trữ vật lý của các chỉ mục B-tree (46,02 MB tại 500K)

Bảng 5.7 tổng hợp dung lượng vật lý thực tế của các chỉ mục trích xuất từ hàm quản trị `pg_relation_size()`.

Bảng 5.7: Chi phí phụ trội lưu trữ vật lý của các chỉ mục B-tree qua các thang đo

Tên chỉ mục	Bài toán áp dụng	Cột đánh chỉ mục	Dung lượng tại 50K (MB)	Dung lượng tại 200K (MB)	Dung lượng tại 500K (MB)	Tỷ trọng trên tổng Index 500K (%)
`idx_properties_province_name`	Q1	`(province_name)`	0,36 MB	1,38 MB	3,40 MB	7,4%

Tên chỉ mục	Bài toán áp dụng	Cột đánh chỉ mục	Dung lượng tại 50K (MB)	Dung lượng tại 200K (MB)	Dung lượng tại 500K (MB)	Tỷ trọng trên tổng Index 500K (%)
`idx_properties_location_price`	Q2	`(province_name, district_name, price, id)`	2,88 MB	11,42 MB	28,53 MB	62,0%
`idx_properties_published_at`	Q3	`(published_at)`	1,09 MB	4,30 MB	10,73 MB	23,3%
`idx_properties_property_type`	Q4	`(property_type_name)`	0,35 MB	1,34 MB	3,36 MB	7,3%
Tổng dung lượng các chỉ mục đo riêng	—	—	**4,68 MB**	**18,44 MB**	**46,02 MB**	**100,0%**
Dung lượng bảng Heap dữ liệu gốc	—	*9 thuộc tính bảng properties*	*~10,1 MB*	*~40,5 MB*	*~100,8 MB*	—
Tỷ lệ Tổng Index / Dung lượng Bảng Heap	—	—	**46,3%**	**45,5%**	**45,6%**	—



Hình 5.3: Chi phí phụ trội lưu trữ (Storage Overhead) của các chỉ mục B-tree

QUAN TRỌNG:

Làm rõ phương pháp luận về con số 46,02 MB:

*Con số **46,02 MB** là **tổng dung lượng của 4 chỉ mục được đo đạc riêng lẻ theo từng cấu hình độc lập** (do giao thức đo lường thực hiện xóa và cách ly chỉ mục trước mỗi cấu hình bằng `reset_indexes.sql`). Con số này **không phải là dung lượng lưu trữ đồng thời được quan sát trong một trạng thái cơ sở dữ liệu đơn lẻ duy nhất** chứa cả 4 chỉ mục cùng lúc. Tuy nhiên, phép cộng gộp này phản ánh chính xác tổng chi phí tài nguyên lưu trữ tối thiểu mà hệ thống*

> *phải gánh chịu nếu quản trị viên quyết định duy trì đồng thời cả 4 chỉ mục này trong môi trường sản xuất.*

Trong đó, chỉ mục Composite `(province_name, district_name, price, id)` chiếm tỷ trọng áp đảo (**28,53 MB**, tương đương **62,0%** tổng dung lượng chỉ mục) do phải lưu trữ cả 4 giá trị thuộc tính kèm con trỏ TID trên từng phần tử lá.

5.4. Tổng hợp đối chiếu và Ma trận chứng tích thực nghiệm

Quá trình kiểm chứng đối chiếu khẳng định:

1. **Kiểm toán 600 kế hoạch thực thi:** Toàn bộ 600 tệp JSON kế hoạch trong `results/benchmark/raw/` được tính toán mã băm SHA-256 lưu trữ tại `results/benchmark/raw_manifest.json`. Script kiểm toán tự động `scripts/verify_benchmark_evidence.py` xác nhận toàn bộ các tệp bằng chứng tồn tại đầy đủ, không bị can thiệp và khớp hoàn toàn với số liệu tổng hợp trong Bảng 5.2.
2. **Kiểm thử tự động phân tầng và Tích hợp PostgreSQL trực tiếp:** Hệ thống kiểm thử tự động gồm **198 test cases** (vượt qua 197 passed, 1 skipped) được phân định nghiêm ngặt thành 3 tầng: Tầng Unit (124 tests), Tầng Integration (60 tests với kiểm thử trực tiếp trên PostgreSQL 18.3 thật ở port 5433 và các thao tác COPY/Rollback/Query) và Tầng End-to-End (14 tests kiểm toán luồng dữ liệu và tính nhất quán số liệu), được tổng hợp chi tiết tại `report/test_validation.md` và `results/validation/test_execution_summary.json`.

CHƯƠNG 6: KẾT LUẬN VÀ KHUYẾN NGHỊ

6.1. Tổng kết các đóng góp chính của đề tài

Đề tài **"Xây dựng bộ dữ liệu về bất động sản, thực hiện các phương pháp tối ưu hiệu năng cho hệ thống khi dữ liệu quá lớn"** đã hoàn thành các mục tiêu nghiên cứu với những đóng góp học thuật và thực tiễn cụ thể:

1. Xây dựng quy trình xử lý dữ liệu chuẩn tắc:

- Xây dựng thành công quy trình nạp luồng kết hợp bộ đệm xáo trộn cố định (`seed = 42`, `buffer_size = 10,000`, `batch_size = 2,000`) từ nguồn `Phuscca/vietnam-real-estates`, xuất ra 500.000 bản ghi chuẩn hóa theo hợp đồng 9 thuộc tính qua ba quy mô thực nghiệm (50.000, 200.000 và 500.000 bản ghi).
- Thiết lập bộ quy tắc làm sạch khách quan, bảo toàn 39.061 tin đăng dạng "giá thỏa thuận" (`price IS NULL`) mà không làm biến dạng phân phối thực tế của thị trường.

2. Khảo sát hiệu năng tầng tiền xử lý dữ liệu:

- Định lượng hóa ưu thế của định dạng cột Parquet (tiết kiệm 55,9% dung lượng đĩa so với CSV) và động cơ Polars (đọc CSV nhanh hơn 40,3 lần và phân tích trong bộ nhớ nhanh hơn 4,0 lần so với Pandas).
- Đánh giá năng lực kiểm soát bộ nhớ của kỹ thuật nạp phân khối (Chunked Streaming) thông qua luồng giám sát RAM thực thời (lấy mẫu RSS 1 ms).

3. Thiết lập khung thực nghiệm tối ưu hóa có kiểm soát trên PostgreSQL 18.3:

- Xây dựng bộ công cụ benchmark tự động tuân thủ các chuẩn mực thực nghiệm khắt khe: cách ly chỉ mục, cập nhật catalog thống kê, 3 lượt warm-up kiểm soát bộ đệm và 20 lượt đo chính thức lấy trung vị và khoảng tứ phân vị IQR.
- Thu thập đầy đủ 600 tệp JSON kế hoạch thực thi thô được kiểm toán mã băm SHA-256, đồng thời thiết lập thủ tục kiểm chứng tương đương ngữ nghĩa truy vấn (`EXCEPT ALL`).

4. Phân tích thực nghiệm đa diện và làm rõ các cơ chế nội tại:

- Định lượng hiệu quả của chỉ mục đơn cột Q1 (giảm 43,5% số khối đệm nạp, tăng tốc 4,04 lần tại 500K).
- Làm sáng tỏ cơ chế Early Termination và triệt tiêu hoàn toàn nút `Sort` của Composite Index trong bài toán Q2, đạt mức tăng tốc 942,37 lần tại 500K.
- Khảo sát thực nghiệm quy luật Sargability qua ma trận 2 × 2 (Q3), chứng minh chỉ mục B-tree bị vô hiệu hóa hoàn toàn khi vị từ chứa hàm ép kiểu, và hiệu năng chỉ xuất hiện khi kết hợp vị từ sargable với chỉ mục tương ứng (tăng tốc 17,97 lần).
- Thiết lập bài toán đối chứng âm Q4 chứng minh chỉ mục B-tree không đem lại lợi ích cho truy vấn tổng hợp toàn bảng, đồng thời đo lường tổng chi phí lưu trữ của 4 chỉ mục đo riêng lẻ theo cấu hình là 46,02 MB (~45,6% kích thước bảng Heap).

6.2. Câu trả lời tường minh cho ba câu hỏi nghiên cứu (RQ1 – RQ3)

Dựa trên toàn bộ kết quả thực nghiệm định lượng, đề tài đưa ra câu trả lời cho ba câu hỏi nghiên cứu:

Trả lời Câu hỏi nghiên cứu 1 (RQ1)

Kết luận RQ1:

- Quy trình nạp luồng kết hợp bộ đệm shuffle cho phép thu thập và chuẩn hóa thành công 500.000 bản ghi bất động sản từ nguồn Hugging Face với tỷ lệ lỗi vi phạm toàn vẹn bị loại bỏ rất thấp (203 bản ghi, chiếm 0,04%).

- Hồ sơ chất lượng dữ liệu phản ánh trung thực đặc trưng thị trường: độ đầy đủ đạt 100% trên 6 trường bắt buộc; tỷ lệ khuyết thiếu có tính phân tầng hành chính rõ rệt: trường `ward_name` khuyết 14,43% (72.150 bản ghi), trường `district_name` khuyết 2,85% (14.235 bản ghi), và trường `price` khuyết 7,81% (39.061 bản ghi dạng giá thỏa thuận).

*- Phân bố không gian tập trung cao độ tại TP. Hồ Chí Minh (37,34%), Hà Nội (34,03%) và Hải Phòng (2,08%). Phân bố loại hình tập trung ở Nhà riêng (46,08%) và Đất (24,12%). Do dữ liệu định lượng có phân phối lệch phải mạnh với nhiều ngoại lệ siêu giá trị, các đại lượng thống kê mô tả vững được xác lập với **trung vị diện tích là 80,0 m²** (IQR = 64,1 m²) và **trung vị giá chào bán là 7,40 tỷ VNĐ** (IQR = 10,20 tỷ VNĐ).*

Trả lời Câu hỏi nghiên cứu 2 (RQ2)

Kết luận RQ2:

- Ở tầng tiền xử lý dữ liệu, định dạng Parquet (41,42 MB) giúp tiết kiệm 55,9% dung lượng lưu trữ trên đĩa so với CSV (93,88 MB) và giảm mức tăng RAM khi nạp vào Pandas từ +241,37 MB xuống +47,95 MB.

- Động cơ Polars thể hiện ưu thế vượt trội: đọc tệp CSV 500K dòng chỉ trong 75,27 ms (nhanh gấp 40,3 lần Pandas read_csv: 3.038,53 ms) và hoàn thành tác vụ lọc kết hợp tổng hợp trong 13,88 ms (nhanh gấp 4,0 lần Pandas: 55,17 ms).

- Kỹ thuật nạp phân khối (Chunked Streaming với kích thước 10.000 dòng) giúp khống chế mức tăng bộ nhớ thực tế: thao tác nạp phân khối chỉ tăng +19,68 MB RAM; thao tác nạp kèm tính toán tổng hợp (Chunked Aggregate) có mức tăng RAM là +21,09 MB (do chi phí tích lũy bảng tổng hợp trung gian). Mặc dù không phải mọi tác vụ Chunked đều có mức tăng dưới 20 MB, kỹ thuật này giúp duy trì dấu chân bộ nhớ ổn định ở ngưỡng ~20-21 MB, thấp hơn 11,4 lần so với nạp toàn bộ tệp, đổi lại thời gian thực thi dài hơn do chi phí vòng lặp.

Trả lời Câu hỏi nghiên cứu 3 (RQ3)

Kết luận RQ3:

- Trong PostgreSQL 18.3, đối với truy vấn lọc đơn cột có độ chọn lọc vừa phải (Q1 - Hải Phòng 2,08%), chỉ mục B-tree chuyển đổi phương thức quét từ `Seq Scan` sang `Bitmap Index Scan + Bitmap Heap Scan`, giảm số khối đệm từ 12.808 xuống 7.236 Shared Hit Blocks, đạt mức tăng tốc từ 4,04 lần (500K) đến 8,21 lần (50K).

- Đối với truy vấn lọc khoảng kết hợp sắp xếp và cắt ngọn (Q2), Composite Index bao phủ đúng thứ tự `(province_name, district_name, price, id)` kích hoạt cơ chế Early Termination, dừng duyệt ngay khi đủ 20 dòng, loại bỏ hoàn toàn nút `Sort` trong kế hoạch thực thi, giảm số khối truy cập bộ đệm từ 12.898 xuống đúng 23 Shared Hit Blocks (giữ cố định qua mọi quy mô dữ liệu), đạt mức tăng tốc 942,37 lần tại 500K. Chi phí thực thi trong trường hợp này bị chặn trên bởi một ngưỡng cận hằng số nhờ k=20 cố định và dữ liệu nằm liên tục trên các nút lá B-tree.

- Đối với truy vấn thời gian (Q3), vị từ non-sargable (`published_at::date`) khiến chỉ mục B-tree bị vô hiệu hóa hoàn toàn (tỷ lệ sử dụng 0,00%, Speedup = 0,95x). Chỉ khi viết lại truy vấn sang khoảng thời gian nửa mở sargable kết hợp chỉ mục B-tree, hệ thống mới kích hoạt được Bitmap Index Scan và đạt mức tăng tốc 17,97 lần tại 500K.

- Trong truy vấn tổng hợp toàn bảng (Q4), chỉ mục B-tree không đem lại lợi ích (Speedup = 0,95x) và bộ lập lịch giữ nguyên `Seq Scan + HashAggregate` do chi phí Random I/O qua chỉ mục lớn hơn quét tuần tự. Tổng dung lượng của 4 chỉ mục đo riêng lẻ theo cấu hình tại 500K là 46,02 MB (chiếm 45,6% dung lượng bảng Heap 100,8 MB), trong đó Composite Index chiếm 62,0% tổng dung lượng chỉ mục.

6.3. Khuyến nghị kỹ thuật thực tiễn cho kỹ sư dữ liệu và DBA

Từ các kết quả nghiên cứu định lượng, nhóm nghiên cứu đề xuất các nguyên tắc kiến trúc và thực hành kỹ thuật:

1. Nguyên tắc thiết kế Composite Index bao phủ truy vấn tìm kiếm phân trang:

- Khi thiết kế chỉ mục cho các chức năng tra cứu danh sách có sắp xếp và phân trang (`LIMIT K`), các cột phải được sắp xếp theo thứ tự nghiêm ngặt:

Index Order = (Cột lọc đẳng thức =, Cột lọc khoảng BETWEEN, Cột ORDER BY, Khóa phá vỡ trùng lặp PK)

- Thứ tự này cho phép hệ thống loại bỏ hoàn toàn nút `Sort` trong bộ nhớ và kích hoạt cơ chế Early Termination, giúp truy vấn đạt thời gian đáp ứng dưới 1 mili-giây.

2. Tuân thủ tính Sargability trong mã SQL tầng ứng dụng:

- Tránh bao bọc cột dữ liệu trong các hàm số học, hàm chuỗi hoặc toán tử ép kiểu trong mệnh đề `WHERE` (như `WHERE col::date = ...` hoặc `WHERE UPPER(name) = ...`).
- Luôn viết lại vị từ dưới dạng so sánh khoảng nửa mở $[T_1, T_2)$ để bộ lập lịch có thể tận dụng chỉ mục B-tree tiêu chuẩn. Trường hợp bắt buộc phải lọc theo biểu thức, cần tạo **Functional Index (Chỉ mục hàm)** chuyên biệt.

3. Tránh tạo chỉ mục B-tree trên các cột chỉ phục vụ tổng hợp toàn bảng:

- Đối với các bài toán báo cáo dashboard tổng hợp dữ liệu trên toàn bảng thiếu điều kiện lọc chọn lọc, việc tạo chỉ mục B-tree đơn thuần không đem lại hiệu quả và gây lãng phí lưu trữ.
- Giải pháp thay thế phù hợp là sử dụng **Materialized Views** được làm mới định kỳ, hoặc thiết kế các bảng tổng hợp tiền kỳ (Aggregate Tables) trong kiến trúc kho dữ liệu.

4. Hiện đại hóa tầng tiền xử lý dữ liệu:

- Ưu tiên lưu trữ dữ liệu phân tích ở định dạng Parquet để tiết kiệm không gian lưu trữ và tăng tốc độ đọc tệp.
- Sử dụng Polars hoặc kỹ thuật Chunked Streaming trong Pandas để kiểm soát đỉnh bộ nhớ RAM, tránh lỗi OOM trong các đường ống ETL/ELT.

6.4. Hạn chế của nghiên cứu

Mặc dù đã thực hiện quy trình thực nghiệm chặt chẽ, nghiên cứu còn tồn tại một số hạn chế khách quan:

1. Môi trường đo lường đơn luồng: Thực nghiệm được tiến hành trên một nút máy chủ cục bộ với mô hình một tiến trình truy vấn tuần tự. Kết quả đo chưa phản ánh các hiện tượng tranh chấp tài nguyên bộ đệm (Buffer Contention), xung đột khóa giao dịch (Lock Contention) và độ trễ mạng trong môi trường phân tán đa người dùng đồng thời.

2. Chưa định lượng trực tiếp độ trễ ghi dữ liệu (Write Penalty): Nghiên cứu đã đo đạc chi phí phụ trội lưu trữ vật lý của các chỉ mục (~45,6% bảng Heap), nhưng chưa tiến hành đo đạc thực nghiệm về độ suy giảm thông lượng của các thao tác ghi dữ liệu (`INSERT`, `UPDATE`, `DELETE`) khi duy trì đồng thời các chỉ mục này.

3. Phạm vi quy mô dữ liệu và giới hạn tài nguyên RAM: Quy mô dữ liệu tối đa trong thực nghiệm là 500.000 bản ghi (~94 MB CSV), nằm hoàn toàn trong dung lượng RAM vật lý 16 GB của máy trạm. Do đó, nghiên cứu chưa có thực nghiệm chứng minh hành vi của hệ thống khi khối lượng dữ liệu vượt quá dung lượng RAM vật lý (hiện tượng tráo đổi bộ nhớ đệm Disk Swapping / Buffer Eviction liên tục).

4. Giới hạn của phép lấy mẫu RSS 1 ms: Phép đo RSS phản ánh bộ nhớ tiến trình do hệ điều hành cấp phát, chịu ảnh hưởng bởi cơ chế quản lý khối trễ của Python runtime và hiện tượng phân mảnh bộ nhớ, do đó chỉ mang tính chất định lượng tương đối.

5. Tính tái lập phụ thuộc vào Dataset Revision nguồn: Do nạp theo chế độ streaming từ repository Hugging Face với `revision: null`, tính tái lập tuyệt đối của dữ liệu phụ thuộc vào việc kho lưu trữ upstream không thay đổi nội dung các tệp dữ liệu nguồn.

6.5. Định hướng phát triển tiếp theo

Trong các giai đoạn nghiên cứu tiếp theo, đề tài có thể được mở rộng theo các hướng:

1. Khảo sát Chỉ mục BRIN cho dữ liệu chuỗi thời gian: Cột `published_at` có tính chất tăng dần theo thời gian vật lý. Việc áp dụng chỉ mục BRIN (Block Range Index) có thể mang lại khả năng nén kích thước chỉ mục xuống vài kilobyte mà vẫn duy trì hiệu năng tìm kiếm khoảng tốt.

2. Phân vùng dữ liệu (Table Partitioning): Mở rộng quy mô dữ liệu lên hàng triệu bản ghi và áp dụng kỹ thuật phân vùng bảng theo thời gian hoặc theo địa bàn hành chính, kết hợp kỹ thuật cắt tỉa phân vùng (Partition Pruning).

3. Kiểm thử tải đa người dùng đồng thời: Sử dụng công cụ `pgbench` để mô phỏng tải truy vấn đồng thời từ hàng trăm kết nối, khảo sát thông lượng giao dịch (TPS) và độ trễ ở các phân vị cao (P_95, P_99).

TÀI LIỆU THAM KHẢO

1. The PostgreSQL Global Development Group (2025). *PostgreSQL 18 Documentation: The PostgreSQL Query Optimizer, Index Types, and B-tree Indexes.* Truy cập tại: https://www.postgresql.org/docs/current/

2. Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G. (1979). Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (pp. 23-34).

3. Stonebraker, M., & Hellerstein, J. M. (2005). *Readings in Database Systems (4th ed.).* MIT Press.

4. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems (3rd ed.).* McGraw-Hill.

5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts (7th ed.).* McGraw-Hill.

6. Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3), 173-189.

7. Winand, M. (2012). *SQL Performance Explained: Everything Developers Need to Know about SQL Performance.* Markus Winand.

8. Apache Software Foundation (2024). *Apache Parquet Documentation: Columnar Storage for Hadoop and Modern Analytic Engines.* Truy cập tại: https://parquet.apache.org/

9. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51-56).

10. Ritchie, R., et al. (2024). *Polars: Blazingly fast DataFrames in Rust, Python, and Node.js.* GitHub repository: https://github.com/pola-rs/polars

11. Tukey, J. W. (1977). *Exploratory Data Analysis.* Addison-Wesley.

12. Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1), 72-101.

13. Hugging Face Datasets Team (2024). *Datasets: A library for easily accessing and sharing datasets.* GitHub repository: https://github.com/huggingface/datasets

14. Phuscca (2024). *vietnam-real-estates dataset.* Hugging Face repository: https://huggingface.co/datasets/Phuscca/vietnam-real-estates

PHỤ LỤC

Phụ lục A: Lược đồ DDL và Ràng buộc toàn vẹn cơ sở dữ liệu

Tệp đặc tả cấu trúc bảng quan hệ `sql/schema.sql`:

```
CREATE TABLE IF NOT EXISTS public.properties (
    id BIGINT PRIMARY KEY,
    name TEXT NOT NULL,
    property_type_name TEXT NOT NULL,
    province_name TEXT NOT NULL,
    district_name TEXT,
    ward_name TEXT,
    price NUMERIC,
    area DOUBLE PRECISION NOT NULL,
    published_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    CONSTRAINT properties_area_positive CHECK (area > 0),
    CONSTRAINT properties_price_positive CHECK (price IS NULL OR price > 0)
);
```

Tập lệnh khởi tạo các chỉ mục B-tree thực nghiệm trong thư mục `sql/benchmark/indexes/`:

```
-- 1. Chỉ mục đơn cột Q1 (q1_province_name.sql)
CREATE INDEX idx_properties_province_name
ON public.properties USING btree (province_name);

-- 2. Chỉ mục tổ hợp đa cột Q2 (q2_location_price.sql)
CREATE INDEX idx_properties_location_price
ON public.properties USING btree (province_name, district_name, price, id);

-- 3. Chỉ mục thời gian Q3 (q3_published_at.sql)
CREATE INDEX idx_properties_published_at
ON public.properties USING btree (published_at);

-- 4. Chỉ mục loại hình bất động sản Q4 (q4_property_type.sql)
CREATE INDEX idx_properties_property_type
ON public.properties USING btree (property_type_name);
```

Phụ lục B: Tập mã nguồn các câu truy vấn Benchmark SQL chuẩn hóa

Đối chiếu trực tiếp với các tệp truy vấn trong thư mục `sql/benchmark/queries/`:

B.1. Bài toán Q1 (`sql/benchmark/queries/q1.sql`)

```
SELECT id, name, price, area
FROM public.properties
WHERE province_name = 'Hải Phòng';
```

B.2. Bài toán Q2 (`sql/benchmark/queries/q2.sql`)

```
SELECT id, name, price, area, published_at
FROM public.properties
WHERE province_name = 'Hà Nội'
  AND district_name = 'Cầu Giấy'
  AND price BETWEEN 4100000000 AND 14300000000
ORDER BY price ASC, id ASC
LIMIT 20;
```

B.3. Bài toán Q3 Truy vấn gốc (`sql/benchmark/queries/q3_original.sql`)

```
SELECT id, name, published_at
FROM public.properties
WHERE published_at::date = DATE '2025-10-15';
```

B.4. Bài toán Q3 Truy vấn viết lại (`sql/benchmark/queries/q3_rewritten.sql`)

```
SELECT id, name, published_at
FROM public.properties
WHERE published_at >= TIMESTAMP '2025-10-15 00:00:00'
  AND published_at <  TIMESTAMP '2025-10-16 00:00:00';
```

B.5. Bài toán Q4 (`sql/benchmark/queries/q4.sql`)

```
SELECT
    property_type_name,
    COUNT(*) AS total,
    ROUND(AVG(price), 0) AS avg_price
FROM public.properties
GROUP BY property_type_name
ORDER BY total DESC;
```

Phụ lục C: Bảng kê mã băm SHA-256 các tệp kế hoạch thực thi thô

Toàn bộ 600 tệp JSON kế hoạch thực thi thô `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` được lưu trữ tại `results/benchmark/raw/{scale}/`. Bảng dưới đây trích dẫn một số mẫu mã băm SHA-256 từ tệp kiểm toán `results/benchmark/raw_manifest.json`:

Quy mô	Cấu hình	Lần chạy	Tên tệp kế hoạch JSON	Mã băm toàn vẹn SHA-256 (64 ký tự hex)
500K	Q1_baseline	Run 01	500000_Q1_baseline_run01.json	7be33f52a78eb04a29a0fef738988a109a0669c3a38ceca9489f61b0c0342217
500K	Q1_indexed	Run 01	500000_Q1_indexed_run01.json	837e3d162bc58d04269d08e33ea2193b2a0c64c7e54f9810b48455fa2ea2c159
500K	Q2_baseline	Run 01	500000_Q2_baseline_run01.json	9b3eb16b8ffec64bc81907cb818ba4dbebf25b045353ff7fc9d0124cbf5e2270
500K	Q2_indexed	Run 01	500000_Q2_indexed_run01.json	ad7be7c7ce1a5e1d4ad3c14a2bf43cf1815b822dc46b6279930f719488aef6a4
500K	Q3_A	Run 01	500000_Q3_A_run01.json	f6a47a16f6b5b4ce799f8d5f303c73fc7827877ce547e1744b827e8a3ffb1d9c
500K	Q3_B	Run 01	500000_Q3_B_run01.json	ca0cb5cf920556f8f533a871fe43b9e4a3b836a9a72df91845bb08f3316223e7
500K	Q3_C	Run 01	500000_Q3_C_run01.json	2e08e8b61cbf7a72bbd8e2840cfad114bf3921074706598501e7415494dcf91f

Quy mô	Cấu hình	Lần chạy	Tên tệp kế hoạch JSON	Mã băm toàn vẹn SHA-256 (64 ký tự hex)
500K	Q3_D	Run 01	500000_Q3_D_run01.json	b68078c59918bfdbe9aa48d6dbfa6cb825a0a38b693259e86a032d84ea235552
500K	Q4_baseline	Run 01	500000_Q4_baseline_run01.json	dbbc3fb4ae3c4078cb18bf332c943ba874aa6794627be71cb87adca246e7f1dc
500K	Q4_indexed	Run 01	500000_Q4_indexed_run01.json	e2a2bafe4db36829ee168fb99564f9bf35a8f273295c52c938368819c9682701